

THE COMPLETE

SQL HANDBOOK

MySQL 8 - Beginner to Advanced

Syntax | Use cases | Examples | Pitfalls

Business analytics | Interview patterns | Practice

TAILORED FOR BUSINESS ANALYST PREPARATION

Advanced SQL, metrics, root-cause analysis, reporting, data quality, and interview reasoning.

Primary dialect: MySQL 8.x

Includes portability notes for PostgreSQL, SQL Server, Oracle, SQLite, BigQuery, and Snowflake.

Generated as a searchable reference and study guide

Table of Contents

Preface: How to Use This Handbook	16
Scope and Dialect	16
Learning Priorities for a Business Analyst	16
A Consistent Practice Schema	16
1. Relational Database Foundations	19
Database Vocabulary	19
Keys	19
Relationship Cardinality	19
Data Grain	20
SQL Is Declarative	20
2. SQL Command Families: DDL, DML, DQL, TCL, and DCL	21
Command Family Summary	21
DDL: Data Definition Language	21
DML: Data Manipulation Language	21
DQL: Data Query Language	22
TCL: Transaction Control Language	22
DCL: Data Control Language	22
3. MySQL Data Types and Type Selection	23

Integer Types	23
Exact and Approximate Numeric Types	23
Character and Text Types	23
Date and Time Types	24
Boolean, ENUM, SET, JSON, and Spatial Types	24
NULL: Unknown Is Not Zero	25
4. SELECT Fundamentals and Query Shape	26
Basic SELECT Syntax	26
Selecting Columns	26
Aliases	26
DISTINCT	26
Expressions	27
5. Filtering with WHERE and Operators	28
Comparison Operators	28
Logical Operators and Precedence	28
IN and NOT IN	28
BETWEEN	28
LIKE, Percent, and Underscore	29
REGEXP	29
IS NULL and Missing Values	29

EXISTS	29
6. Sorting, LIMIT, OFFSET, and Pagination	30
ORDER BY	30
LIMIT	30
OFFSET Explained	30
Keyset Pagination	30
7. CASE and Conditional Logic	32
Searched CASE	32
Simple CASE	32
Bucketing	32
Conditional Aggregation	32
8. Built-in Functions and Expressions	34
String Functions	34
Numeric Functions	34
Date and Time Functions	34
CAST and CONVERT	35
COALESCE, IFNULL, NULLIF, and IF	35
9. Aggregate Functions, GROUP BY, and HAVING	36
Core Aggregate Functions	36
GROUP BY	36

WHERE versus HAVING	36
COUNT DISTINCT	37
Ratios and Percentages	37
Weighted Average	37
Avoiding Double Counting	38
GROUP_CONCAT and WITH ROLLUP	38
10. Joins: Combining Related Tables	39
INNER JOIN	39
LEFT JOIN	39
RIGHT JOIN	39
FULL OUTER JOIN	40
CROSS JOIN	40
Self Join	40
Non-Equi and Range Joins	40
ON versus WHERE with Outer Joins	40
Semi-Join with EXISTS	41
Anti-Join with NOT EXISTS	41
Many-to-Many Joins	41
Diagnosing Join Multiplication	42
11. Set Operations	43

UNION and UNION ALL	43
INTERSECT and EXCEPT Portability	43
12. Subqueries	44
Scalar Subquery	44
Single-Column Multi-Row Subquery	44
Subquery in SELECT	44
Derived Table in FROM	44
Correlated Subquery	45
ANY, SOME, and ALL	45
Comparing One Row's Value to the Same Column	45
When to Use Each Form	45
13. Common Table Expressions (CTEs)	46
Basic CTE	46
Multiple CTEs	46
CTE versus Alternatives	46
Recursive CTE	47
14. Window Functions	48
Window Syntax	48
Aggregate versus Window	48
ROW_NUMBER, RANK, and DENSE_RANK	48

Top N per Group	99
LAG and LEAD	49
Running Total	49
Moving Average	50
FIRST_VALUE, LAST_VALUE, and NTH_VALUE	50
NTILE, PERCENT_RANK, and CUME_DIST	50
Named Windows	50
Deduplicate to Latest Row	51
15. Business Analytics SQL Patterns	52
Metric Definition Template	52
Revenue and Order Value	52
Average Order Value	52
Conversion Rate and Funnel	53
Root-Cause Segmentation	53
Period-over-Period Growth	54
Share of Total	54
Pareto / 80-20 Analysis	55
Cohort Retention	55
Repeat Purchase and Reactivation	56
RFM Segmentation	56

Sessionization	56
Gaps and Islands	57
Pivot and Unpivot	57
Date Spine and Zero-Filling	58
16. INSERT, UPDATE, DELETE, and Safe Data Changes	59
INSERT	59
Upsert	59
UPDATE	59
DELETE	60
DELETE versus TRUNCATE versus DROP	60
17. Table Design, Constraints, and Normalization	61
CREATE TABLE Anatomy	61
Constraints	61
Generated Columns	61
Temporary Tables	62
Normalization	62
Denormalization	62
OLTP versus OLAP	62
Star Schema	62
18. Transactions, ACID, and Concurrency	64

ACID	64
Transaction Commands	64
Isolation Levels and Anomalies	64
MVCC and Locking Reads	65
Deadlocks	65
Optimistic Concurrency	65
19. Views, Stored Programs, Triggers, and Events	66
Views	66
Stored Procedures	66
Stored Functions	67
Triggers	67
Events	68
DEFINER and SQL SECURITY	68
20. JSON in MySQL 8	69
Creating and Inserting JSON	69
Extracting Values	69
Modifying JSON	69
Searching JSON	69
JSON_TABLE	70
JSON Aggregation	70

Indexing JSON	70
21. Indexes and the InnoDB Storage Model	71
InnoDB Primary and Secondary Indexes	71
Creating Indexes	71
Composite Index and Leftmost Prefix	71
Equality, Range, and Ordering	71
Covering Index	72
Selectivity and Cardinality	72
Specialized Indexes	72
Sargability	72
22. EXPLAIN and Query Optimization	74
EXPLAIN	74
EXPLAIN ANALYZE	74
FORMAT JSON	74
Optimization Workflow	74
Common Improvements	75
Statistics and Histograms	75
Partitioning Awareness	75
Operational Tools	75
23. Security, Privacy, and Safe SQL	76

Preventing SQL Injection	76
Least Privilege	76
Sensitive Data	76
Backups and Recovery Awareness	76
Prepared Statement Example	76
24. Data Quality, Debugging, and Reconciliation	78
Incremental Query Development	78
Duplicate Detection	78
Missing and Invalid Values	78
Referential Gaps	78
Grain and Fanout Checks	79
Reconciliation	79
Date and Time Debugging	79
SQL Errors and Warnings	80
Data Quality Scorecard	80
25. MySQL Dialect Differences and Portability	81
Common Differences	81
Important MySQL-Specific Behavior	81
Portable Style	81
26. Advanced Analytical Patterns	83

Median in MySQL 8	83
Percentile by Nearest Rank	83
As-Of Join / Effective-Dated Dimension	83
Overlapping Date Ranges	84
First and Last Event per Entity	84
Time Between Events	84
Relational Division: Customers Who Bought Every Required Cate... ..	84
Running Balance with Signed Transactions	85
Market Basket Pairs	85
A/B Experiment Summary	85
Snapshot versus Event Data	86
27. SQL Interview Patterns with Solutions	87
Pattern 1: Nth Highest Value	87
Pattern 2: Above Group Average	87
Pattern 3: Top Three per Category	87
Pattern 4: Customers with No Orders	88
Pattern 5: Latest Record per Entity	88
Pattern 6: Find and Safely Remove Duplicates	88
Pattern 7: First and Repeat Purchase	88
Pattern 8: Consecutive Days	89

Pattern 9: Month-over-Month Growth	89
Pattern 10: Percent of Total	89
Pattern 11: Running Total	89
Pattern 12: Retention	90
Pattern 13: Funnel Conversion	90
Pattern 14: Join Inflation	90
Pattern 15: Find Missing Sequence Values	90
Pattern 16: Overlapping Intervals	90
Pattern 17: Conditional KPI Report	90
Pattern 18: Recursive Hierarchy	91
Pattern 19: Median	91
Pattern 20: Data Validation	91
Interview Communication Template	91
28. Business Analyst Cases for E-Commerce	92
Case 1: Orders Dropped 12% Yesterday	92
Case 2: Conversion Increased but Returns Also Increased	92
Case 3: Discount Campaign Evaluation	93
Case 4: Seller Activation Declined	93
Case 5: Delivery Time Increased in One Region	93
Case 6: Build a Daily Pod Dashboard	93

Insight-to-Action Template	94
29. Practice Curriculum and Exercises	95
Level 1: Fundamentals	95
Level 2: Aggregation and Joins	95
Level 3: CTEs, Subqueries, and Windows	95
Level 4: Analytics Cases	96
Level 5: Design and Performance	96
Capstone Project	96
Self-Scoring Rubric	96
30. Master Syntax Cheat Sheet	98
Core SELECT	98
Filter Templates	98
JOIN Decision Table	98
Aggregation Templates	98
CTE Template	99
Recursive CTE Template	99
Window Templates	99
Top N per Group	99
Latest Row per Entity	100
Period-over-Period	100

Safe Transaction	100
DDL Quick Reference	100
DML Quick Reference	100
Common Function Index	101
Destructive Statement Safety Checklist	101
Interview Clarification Checklist	101
31. Common Mistakes and Their Fixes	103
32. Glossary	104
33. Reference and Continued Practice	106
Primary Documentation	106
Learning and Practice Path	106
Final Readiness Standard	106

Preface: How to Use This Handbook

This handbook is a complete, practical SQL reference for a learner starting from zero and progressing toward advanced Business Analyst work. It uses MySQL 8.x as the primary dialect because that matches the MySQL learning path shown in the attached Alex The Analyst course screenshots. It is tailored to the attached Meesho Business Analyst description, which emphasizes advanced SQL, reporting, pod-level metric ownership, root-cause analysis, structured problem solving, and insight communication; the analytics and interview chapters are therefore intentionally extensive.

SQL is learned by typing and reasoning, not by reading alone. For each topic, follow this loop: understand the output grain, type the example, predict the output, run it, change one condition, and explain why the result changed. Never copy a solution that you cannot explain line by line.

NOTE SQL is declarative: you describe the result you want, while the database optimizer decides how to produce it. Correct reasoning about tables, relationships, grain, NULLs, and duplicates matters more than memorizing keywords.

Scope and Dialect

Examples assume MySQL 8.0 or later. Standard SQL concepts are identified where useful, and the portability chapter compares PostgreSQL, SQL Server, Oracle, SQLite, BigQuery, and Snowflake. Feature availability can differ between minor versions, so verify production behavior against the exact installed version.

This is a learning and interview guide, not a substitute for your organization's data dictionary, metric definitions, access policies, or production change process. Examples use fictitious e-commerce data and generic names.

Learning Priorities for a Business Analyst

Priority	Competency	Evidence you should produce
1	Query correctness	Correct result at the requested grain, including edge cases
2	Joins and aggregation	No accidental row multiplication or inflated metrics
3	Advanced analytics	CTEs, windows, retention, funnels, trends, and ranking
4	Validation	Reconcile totals, inspect NULLs, duplicates, and unmatched keys
5	Communication	State metric definition, assumptions, finding, action, and risk
6	Performance awareness	Read EXPLAIN and propose evidence-based indexes

A Consistent Practice Schema

Most examples use this simplified commerce model:

```
CREATE DATABASE IF NOT EXISTS commerce_lab;
USE commerce_lab;
```

SQL


```

CREATE TABLE customers (
  customer_id    BIGINT PRIMARY KEY AUTO_INCREMENT,
  customer_name  VARCHAR(100) NOT NULL,
  signup_at     DATETIME NOT NULL,
  city          VARCHAR(80),
  acquisition_channel VARCHAR(40),
  status        ENUM('active','inactive','blocked') DEFAULT 'active'
);

CREATE TABLE categories (
  category_id    INT PRIMARY KEY AUTO_INCREMENT,
  category_name  VARCHAR(80) NOT NULL UNIQUE
);

CREATE TABLE products (
  product_id    BIGINT PRIMARY KEY AUTO_INCREMENT,
  category_id    INT NOT NULL,
  product_name  VARCHAR(150) NOT NULL,
  list_price    DECIMAL(12,2) NOT NULL,
  unit_cost     DECIMAL(12,2) NOT NULL,
  active_flag   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_product_category
    FOREIGN KEY (category_id) REFERENCES categories(category_id),
  CONSTRAINT chk_product_price CHECK (list_price >= 0 AND unit_cost >= 0)
);

CREATE TABLE orders (
  order_id      BIGINT PRIMARY KEY AUTO_INCREMENT,
  customer_id   BIGINT NOT NULL,
  order_at      DATETIME NOT NULL,
  status        ENUM('placed','paid','shipped','delivered','cancelled','returned') NOT
    NULL,
  payment_method VARCHAR(30),
  shipping_city VARCHAR(80),
  CONSTRAINT fk_order_customer
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  INDEX idx_orders_customer_date (customer_id, order_at),
  INDEX idx_orders_date_status (order_at, status)
);

CREATE TABLE order_items (
  order_id      BIGINT NOT NULL,
  product_id    BIGINT NOT NULL,
  quantity      INT NOT NULL,
  unit_price    DECIMAL(12,2) NOT NULL,
  discount_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id),
  CHECK (quantity > 0),
  CHECK (unit_price >= 0 AND discount_amount >= 0)
);

CREATE TABLE payments (
  payment_id    BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id      BIGINT NOT NULL,
  payment_at    DATETIME,
  amount        DECIMAL(12,2) NOT NULL,
  payment_status ENUM('initiated','successful','failed','refunded') NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  INDEX idx_payments_order_status (order_id, payment_status)
);

```

SQL

```
CREATE TABLE events (  
  event_id      BIGINT PRIMARY KEY AUTO_INCREMENT,  
  customer_id   BIGINT,  
  session_id    VARCHAR(64) NOT NULL,  
  event_at      DATETIME(6) NOT NULL,  
  event_name     VARCHAR(50) NOT NULL,  
  product_id    BIGINT,  
  properties     JSON,  
  INDEX idx_events_name_date (event_name, event_at),  
  INDEX idx_events_customer_date (customer_id, event_at)  
);
```

The grain of each table is crucial: one row per customer, product, order, order-product combination, payment attempt, or event. Write the grain before writing an analytical query.

1. Relational Database Foundations

Database Vocabulary

A database is an organized collection of data. A database management system such as MySQL stores the data, enforces rules, handles concurrent access, and executes SQL. A schema is the logical structure containing tables and other objects. In MySQL, DATABASE and SCHEMA are effectively synonyms.

A table represents one entity or event type. A row is one instance at the table's declared grain. A column is one attribute with a data type. A constraint is a rule enforced by the database. A relationship connects tables through key values.

SQL

```
SHOW DATABASES;
CREATE DATABASE commerce_lab;
USE commerce_lab;
SHOW TABLES;
DESCRIBE orders;
SHOW CREATE TABLE orders;
```

INFORMATION_SCHEMA exposes metadata as queryable tables:

SQL

```
SELECT table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'commerce_lab'
ORDER BY table_name;

SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'commerce_lab'
  AND table_name = 'orders'
ORDER BY ordinal_position;
```

Keys

Key type	Meaning	Example
Primary key	Chosen unique, non-NULL row identifier	orders.order_id
Foreign key	Value that references a key in another table	orders.customer_id
Candidate key	Any minimal set that could uniquely identify a row	email, if guaranteed unique
Alternate key	Candidate key not chosen as primary	a unique order reference
Composite key	Key made from multiple columns	(order_id, product_id)
Natural key	Identifier with business meaning	country code
Surrogate key	Artificial identifier	auto-increment integer

A primary key is not the same as a row number. It should remain stable and unique. A foreign key can repeat because one customer can have many orders.

Relationship Cardinality

One-to-one: one row in A relates to at most one row in B. Example: one customer and one optional customer profile. Enforce with a unique foreign key.

One-to-many: one customer has many orders. Put the foreign key on the many side: `orders.customer_id`.

Many-to-many: many orders contain many products. Resolve with a bridge table, `order_items`, whose key typically combines both parent keys and may contain relationship attributes such as quantity and price.

Data Grain

Grain answers: What does one row represent? If `orders` is one row per order and `order_items` is one row per order-product, joining them changes the row count to item grain. Summing an order-level amount after that join can multiply the amount by the number of items.

Before every query, write:

- Source grain: one row per what?
- Desired output grain: one row per what?
- Measure: what is being counted or summed?
- Population: which rows qualify?
- Time basis and time zone: which date controls inclusion?

SQL Is Declarative

You write what result is needed, not a procedural loop over rows. SQL uses set-oriented operations. The optimizer can reorder physical work while preserving logical meaning.

The logical processing order of a basic query is:

1. FROM and JOIN/ON
2. WHERE
3. GROUP BY
4. aggregate calculations
5. HAVING
6. SELECT expressions and aliases
7. DISTINCT
8. ORDER BY
9. LIMIT and OFFSET

TEXT

This explains why a SELECT alias is usually available in ORDER BY but not in WHERE: WHERE is logically evaluated earlier.

2. SQL Command Families: DDL, DML, DQL, TCL, and DCL

Command Family Summary

Family	Purpose	Main commands
DDL	Define structure	CREATE, ALTER, RENAME, TRUNCATE, DROP
DML	Insert or modify rows	INSERT, UPDATE, DELETE
DQL	Retrieve rows	SELECT
TCL	Control transactions	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT
DCL	Control access	GRANT, REVOKE, roles and users

These categories are useful for learning, although vendors and books sometimes classify them slightly differently.

DDL: Data Definition Language

SQL

```
CREATE TABLE warehouses (
  warehouse_id INT PRIMARY KEY AUTO_INCREMENT,
  warehouse_name VARCHAR(100) NOT NULL,
  city VARCHAR(80) NOT NULL,
  opened_on DATE,
  capacity_units INT DEFAULT 0 CHECK (capacity_units >= 0)
);

ALTER TABLE warehouses ADD COLUMN active_flag BOOLEAN NOT NULL DEFAULT TRUE;
ALTER TABLE warehouses MODIFY COLUMN warehouse_name VARCHAR(150) NOT NULL;
ALTER TABLE warehouses RENAME COLUMN city TO warehouse_city;
RENAME TABLE warehouses TO fulfilment_centers;
TRUNCATE TABLE fulfilment_centers;
DROP TABLE fulfilment_centers;
```

TRUNCATE removes all rows efficiently but preserves the table definition. DROP removes the object. In MySQL, DDL commonly causes implicit commits; do not assume it can be rolled back like ordinary DML.

DML: Data Manipulation Language

SQL

```
INSERT INTO categories (category_name)
VALUES ('Electronics'), ('Fashion'), ('Home');

UPDATE products
SET list_price = ROUND(list_price * 0.95, 2)
WHERE category_id = 2;

DELETE FROM events
WHERE event_at < '2024-01-01';
```

Always preview an UPDATE or DELETE predicate with SELECT first. In production, use a transaction when practical and verify affected row counts.

DQL: Data Query Language

SQL

```
SELECT order_id, customer_id, order_at, status
FROM orders
WHERE order_at >= '2026-01-01'
ORDER BY order_at DESC
LIMIT 100;
```

TCL: Transaction Control Language

SQL

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500.00 WHERE account_id = 10;
UPDATE accounts SET balance = balance + 500.00 WHERE account_id = 20;

COMMIT;
-- Use ROLLBACK instead if validation fails.
```

Transactions are explained in depth later.

DCL: Data Control Language

SQL

```
CREATE ROLE analyst_readonly;
GRANT SELECT ON commerce_lab.* TO analyst_readonly;
CREATE USER 'analyst_user'@'10.%' IDENTIFIED BY 'use-a-secret-manager';
GRANT analyst_readonly TO 'analyst_user'@'10.%';
SET DEFAULT ROLE analyst_readonly TO 'analyst_user'@'10.%';
SHOW GRANTS FOR 'analyst_user'@'10.%';
REVOKE SELECT ON commerce_lab.payments FROM analyst_readonly;
```

Use least privilege. Analysts usually need read access, not unrestricted write or administrative rights.

3. MySQL Data Types and Type Selection

Choosing a type is a correctness decision, not only a storage decision. A type controls valid values, precision, comparisons, ordering, indexes, and function behavior.

Integer Types

Type	Signed range, approximately	Typical use
TINYINT	-128 to 127	small status code or boolean-like flag
SMALLINT	-32 thousand to 32 thousand	small counts
MEDIUMINT	-8.3 million to 8.3 million	specialized counters
INT / INTEGER	-2.1 billion to 2.1 billion	common IDs and counts
BIGINT	about +/-9.22 quintillion	high-volume IDs and event counts

UNSIGNED removes negative values and roughly doubles the positive range. Choose enough range for future growth, but do not use BIGINT blindly everywhere.

```
quantity SMALLINT UNSIGNED NOT NULL,
customer_id BIGINT UNSIGNED NOT NULL
```

SQL

Exact and Approximate Numeric Types

Use DECIMAL(p,s) for money and values requiring exact decimal arithmetic. p is total digits; s is digits after the decimal point.

```
price DECIMAL(12,2)  -- up to 10 digits before and 2 after decimal
rate  DECIMAL(7,6)   -- values such as 0.123456
```

SQL

FLOAT and DOUBLE are approximate binary floating-point types. They are suitable for scientific measurements, but equality checks and currency totals can surprise you.

```
SELECT 0.1 + 0.2;                -- representation may be approximate
SELECT CAST(0.1 AS DECIMAL(10,2))
       + CAST(0.2 AS DECIMAL(10,2)); -- exact decimal result
```

SQL

Use ROUND(value, decimal_places) for presentation or defined business rounding. Keep full precision during intermediate calculations unless the business rule says otherwise.

Character and Text Types

Type	Behavior	Typical use
CHAR(n)	fixed length, padded semantics	fixed codes such as ISO country code
VARCHAR(n)	variable length up to declared maximum	names, labels, references
TINYTEXT/TEXT/MEDIUMTEXT/LONGTEXT	large text families	notes and descriptions
BINARY/VARBINARY	raw bytes with fixed/variable size	hashes or binary identifiers
BLOB families	binary large objects	files; often better stored externally

Prefer utf8mb4 for full Unicode. A collation controls comparison and sorting, including case and accent sensitivity.

```
CREATE TABLE labels (
  label_id INT PRIMARY KEY,
  label_text VARCHAR(100)
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_0900_ai_ci
);
```

SQL

ai_ci generally means accent-insensitive and case-insensitive. Collation affects equality, joins, uniqueness, and duplicate detection.

LENGTH(text) returns bytes; CHAR_LENGTH(text) returns characters.

```
SELECT LENGTH('SQL'), CHAR_LENGTH('SQL');
```

SQL

Date and Time Types

Type	Stores	Important note
DATE	calendar date	no time or zone
TIME	time or duration-like value	supports a broad hour range
DATETIME[(fsp)]	date and time	no automatic time-zone conversion
TIMESTAMP[(fsp)]	instant represented via session	narrower range than DATETIME
	time zone	
YEAR	year value	specialized and rarely necessary

Use DATETIME for business-local values or broad date range. Use TIMESTAMP when MySQL's time-zone conversion behavior fits your design. Store a defined time zone, often UTC, and convert for reporting.

```
SET time_zone = '+00:00';
SELECT NOW(), CURRENT_DATE(), CURRENT_TIMESTAMP(6);

SELECT CONVERT_TZ(order_at, '+00:00', '+05:30') AS order_at_india
FROM orders;
```

SQL

Do not store dates in formatted text columns. Text dates sort incorrectly unless strictly ISO-formatted and make validation and arithmetic difficult.

Boolean, ENUM, SET, JSON, and Spatial Types

MySQL treats BOOLEAN/BOOL as a synonym for TINYINT(1). Use constraints or application validation if only 0 and 1 are valid.

ENUM restricts a column to one value from a list. It is compact but schema changes are needed to add values, and ordering can follow internal enumeration order rather than lexical order. A lookup table is often more flexible.

SET permits multiple members from a predefined list in one column. It violates simple normalization and is usually inferior to a bridge table.

JSON stores validated JSON documents and supports path extraction and transformation. It should not replace properly modeled relational columns used frequently for joins and constraints.

Spatial types such as POINT, LINESTRING, and POLYGON support geographic or geometric work. They are outside typical BA interviews but useful to recognize.

NULL: Unknown Is Not Zero

NULL represents missing, unknown, or not applicable. It is not equal to anything, including another NULL.

SQL

```
-- Wrong: comparison evaluates to UNKNOWN, not TRUE.
SELECT * FROM customers WHERE city = NULL;

-- Correct:
SELECT * FROM customers WHERE city IS NULL;
SELECT * FROM customers WHERE city IS NOT NULL;
```

SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. WHERE keeps only TRUE rows. Comparisons involving NULL normally produce UNKNOWN.

MySQL's null-safe equality operator <=> treats two NULLs as equal:

SQL

```
SELECT NULL = NULL;      -- NULL
SELECT NULL <=> NULL;    -- 1
SELECT 5 <=> 5;          -- 1
```

Useful functions:

SQL

```
COALESCE(city, 'Unknown')      -- first non-NULL argument
IFNULL(city, 'Unknown')        -- MySQL two-argument convenience
NULLIF(quantity, 0)            -- NULL when arguments are equal
revenue / NULLIF(order_count, 0) -- avoids division by zero
```

COUNT() counts rows. COUNT(column) counts non-NULL values only. Most aggregates ignore NULL; if all input values are NULL, many return NULL.

4. SELECT Fundamentals and Query Shape

Basic SELECT Syntax

```
SELECT [DISTINCT]
    expression AS alias,
    another_column
FROM table_name AS t
WHERE row_condition
GROUP BY grouping_columns
HAVING group_condition
ORDER BY sort_expression [ASC|DESC]
LIMIT row_count OFFSET rows_to_skip;
```

SQL

Only SELECT is mandatory when selecting literals:

```
SELECT 2 + 3 AS result,
    'Business Analyst' AS role,
    CURRENT_DATE() AS run_date;
```

SQL

Selecting Columns

```
SELECT customer_id, customer_name, city
FROM customers;

SELECT *
FROM customers;
```

SQL

SELECT is useful for exploration but usually poor in production queries: it reads unnecessary data, hides dependencies, can break consumers when columns change, and can create duplicate column names in joins.

Aliases

```
SELECT
    o.order_id,
    o.order_at AS order_timestamp,
    oi.quantity * oi.unit_price - oi.discount_amount AS net_item_value
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id;
```

SQL

Use aliases that make business meaning obvious. AS is optional for table aliases and often optional for column aliases, but explicit AS improves readability.

DISTINCT

```
SELECT DISTINCT shipping_city
FROM orders
ORDER BY shipping_city;
```

SQL

For multiple columns, DISTINCT applies to the entire selected combination:

SQL

```
SELECT DISTINCT shipping_city, payment_method
FROM orders;
```

Do not use `DISTINCT` automatically to hide duplicates produced by a wrong join. First determine why duplicates exist and whether multiple rows are legitimate at the output grain.

Expressions

SQL

```
SELECT
  order_id,
  quantity,
  unit_price,
  discount_amount,
  quantity * unit_price AS gross_value,
  quantity * unit_price - discount_amount AS net_value
FROM order_items;
```

SQL expressions can contain columns, literals, operators, functions, and `CASE`. Parentheses make precedence explicit.

5. Filtering with WHERE and Operators

Comparison Operators

Operator | Meaning

-----+	
=	equal
<> or !=	not equal
>, >=	greater than, greater than or equal
<, <=	less than, less than or equal
<=>	MySQL null-safe equality

SQL

```
SELECT * FROM products WHERE list_price >= 1000;
SELECT * FROM orders WHERE status <> 'cancelled';
```

Logical Operators and Precedence

NOT is evaluated before AND, and AND before OR. Use parentheses whenever mixed logic could be misunderstood.

SQL

```
-- Delivered orders from Delhi, or any returned order:
WHERE (status = 'delivered' AND shipping_city = 'Delhi')
    OR status = 'returned';
```

Without parentheses, a reviewer may interpret the rule incorrectly even if precedence produces the intended result.

IN and NOT IN

SQL

```
SELECT * FROM orders
WHERE status IN ('paid', 'shipped', 'delivered');

SELECT * FROM products
WHERE category_id NOT IN (1, 4, 7);
```

IN (1,5) means exactly 1 or 5; it does not mean every number from 1 through 5. Use BETWEEN 1 AND 5 for that inclusive range.

NOT IN is dangerous when a subquery can return NULL. Prefer NOT EXISTS for anti-joins.

BETWEEN

BETWEEN low AND high is inclusive at both ends:

SQL

```
WHERE list_price BETWEEN 500 AND 1000
-- equivalent to list_price >= 500 AND list_price <= 1000
```

For timestamps, use a half-open interval instead of BETWEEN:

SQL

```
WHERE order_at >= '2026-01-01'
    AND order_at < '2026-02-01';
```

This includes every precision within January and avoids guessing the final time, such as 23:59:59.999999.

LIKE, Percent, and Underscore

% matches zero or more characters; _ matches exactly one character.

```
WHERE product_name LIKE 'Toy Story%' -- Toy Story, Toy Story 2, etc.
WHERE product_name LIKE 'WALL-_'    -- exactly one character after WALL-
WHERE product_name LIKE '%phone%'   -- contains phone anywhere
```

SQL

To match a literal wildcard, declare an escape character:

```
WHERE coupon_code LIKE 'SAVE\_%' ESCAPE '\\';
```

SQL

A leading wildcard such as %phone usually prevents efficient B-tree index seeking. Consider full-text search for large text-search workloads.

REGEXP

MySQL regular expressions support more complex patterns:

```
SELECT product_name
FROM products
WHERE product_name REGEXP '^(Pro|Premium)[[:space:]]';
```

SQL

Use regular expressions only when simpler predicates cannot express the rule; they can be harder to maintain and index.

IS NULL and Missing Values

```
SELECT customer_id, customer_name
FROM customers
WHERE city IS NULL OR TRIM(city) = '';
```

SQL

NULL, empty string, whitespace, 0, and a placeholder such as 'Unknown' are different states. Decide which states represent missing data in the business definition.

EXISTS

EXISTS asks whether a subquery returns at least one row:

```
SELECT c.customer_id, c.customer_name
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.customer_id
    AND o.status = 'delivered'
);
```

SQL

The selected literal inside EXISTS is irrelevant; SELECT 1 communicates intent.

6. Sorting, LIMIT, OFFSET, and Pagination

ORDER BY

```
SELECT order_id, order_at, status
FROM orders
ORDER BY order_at DESC, order_id DESC;
```

SQL

ASC is the default. Add a unique tie-breaker such as `order_id` so repeated executions return deterministic order when timestamps tie.

MySQL generally places NULL first in ascending order and last in descending order. To control it explicitly:

```
ORDER BY city IS NULL, city ASC; -- non-NULL first, then city
```

SQL

LIMIT

```
SELECT product_id, product_name, list_price
FROM products
ORDER BY list_price DESC, product_id
LIMIT 5;
```

SQL

LIMIT is applied after sorting. Without ORDER BY, "first five" has no reliable business meaning.

MySQL supports both forms:

```
LIMIT 5 OFFSET 10; -- skip 10, return next 5
LIMIT 10, 5;      -- MySQL form: offset, row_count
```

SQL

Prefer LIMIT 5 OFFSET 10 because the words reduce argument-order mistakes.

OFFSET Explained

OFFSET *n* discards the first *n* rows of the ordered result, then LIMIT returns rows. For page size 20:

```
Page 1: LIMIT 20 OFFSET 0
Page 2: LIMIT 20 OFFSET 20
Page 3: LIMIT 20 OFFSET 40
OFFSET = (page_number - 1) * page_size
```

TEXT

Deep offsets can be slow because the database may still scan and discard many rows. They can also produce inconsistent pages when rows are inserted between requests.

Keyset Pagination

For large, changing data, continue from the last seen ordered key:

```
-- First page
SELECT order_id, order_at
FROM orders
```

SQL

SQL

```
ORDER BY order_at DESC, order_id DESC
LIMIT 20;

-- Next page, using the previous page's final values
SELECT order_id, order_at
FROM orders
WHERE order_at < '2026-07-18 10:30:00'
      OR (order_at = '2026-07-18 10:30:00' AND order_id < 987654)
ORDER BY order_at DESC, order_id DESC
LIMIT 20;
```

This is often called cursor or seek pagination.

7. CASE and Conditional Logic

Searched CASE

```

SELECT
    order_id,
    status,
    CASE
        WHEN status = 'delivered' THEN 'Completed'
        WHEN status IN ('cancelled', 'returned') THEN 'Lost or reversed'
        ELSE 'In progress'
    END AS lifecycle_group
FROM orders;

```

SQL

Conditions are checked top to bottom; the first TRUE branch wins. Put specific conditions before broad conditions.

Simple CASE

```

CASE status
    WHEN 'delivered' THEN 'Completed'
    WHEN 'cancelled' THEN 'Cancelled'
    ELSE 'Other'
END

```

SQL

Simple CASE compares one expression for equality. Searched CASE supports ranges and compound logic.

Bucketing

```

SELECT
    product_id,
    list_price,
    CASE
        WHEN list_price < 500 THEN 'Budget'
        WHEN list_price < 2000 THEN 'Mid-range'
        ELSE 'Premium'
    END AS price_band
FROM products;

```

SQL

The second condition means $500 \leq \text{list_price} < 2000$ because the first branch already captured lower values.

Conditional Aggregation

```

SELECT
    DATE(order_at) AS order_date,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN status = 'delivered' THEN 1 ELSE 0 END) AS delivered_orders,
    SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
    100.0 * SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(*), 0) AS cancellation_rate_pct
FROM orders

```

SQL


```
GROUP BY DATE(order_at);
```

Conditional aggregation is the standard SQL technique for pivot-like KPI columns.

8. Built-in Functions and Expressions

String Functions

Function	Example	Purpose
CONCAT	CONCAT(first, ' ', last)	join strings
CONCAT_WS	CONCAT_WS(' ', city, state)	join with separator, skipping NULL
UPPER / LOWER	LOWER(email)	case conversion
TRIM	TRIM(customer_name)	remove surrounding spaces
LTRIM / RTRIM	LTRIM(text)	remove one side's spaces
LENGTH	LENGTH(text)	byte length
CHAR_LENGTH	CHAR_LENGTH(text)	character count
SUBSTRING	SUBSTRING(code, 1, 3)	extract part
LEFT / RIGHT	RIGHT(phone, 4)	take end characters
REPLACE	REPLACE(text, 'old', 'new')	replace occurrences
LOCATE	LOCATE('@', email)	find substring position
LPAD / RPAD	LPAD(id, 8, '0')	pad to length

```

SELECT
  customer_id,
  TRIM(customer_name) AS clean_name,
  LOWER(TRIM(customer_name)) AS normalized_name,
  CONCAT('CUST-', LPAD(customer_id, 8, '0')) AS display_id
FROM customers;

```

SQL

CONCAT returns NULL if any argument is NULL. CONCAT_WS skips NULL arguments, making it useful for optional address parts.

Numeric Functions

```

SELECT
  ABS(-15) AS absolute_value,
  ROUND(123.4567, 2) AS rounded,
  CEIL(12.1) AS ceiling_value,
  FLOOR(12.9) AS floor_value,
  MOD(17, 5) AS remainder,
  POWER(2, 8) AS power_value,
  SQRT(81) AS root_value;

```

SQL

Be explicit about the required rounding rule. ROUND, TRUNCATE, CEIL, and FLOOR answer different business questions.

Date and Time Functions

```

SELECT
  NOW() AS current_datetime,
  CURRENT_DATE() AS current_date,
  DATE(order_at) AS order_date,
  YEAR(order_at) AS order_year,
  MONTH(order_at) AS order_month,
  EXTRACT(QUARTER FROM order_at) AS order_quarter,
  DATE_ADD(order_at, INTERVAL 7 DAY) AS plus_seven_days,
  DATE_SUB(order_at, INTERVAL 1 MONTH) AS minus_one_month,
  DATEDIFF(CURRENT_DATE(), DATE(order_at)) AS age_days,

```

SQL

```

                                SQL
    TIMESTAMPDIFF(HOUR, order_at, NOW()) AS age_hours
FROM orders;
```

Month grouping must include year:

```

                                SQL
-- Safe month key
DATE_FORMAT(order_at, '%Y-%m')

-- Better for date arithmetic: first day of month
DATE_SUB(DATE(order_at), INTERVAL DAYOFMONTH(order_at) - 1 DAY)
```

Do not group only by MONTH(order_at) across multiple years.

Parsing and formatting:

```

                                SQL
SELECT STR_TO_DATE('18/07/2026', '%d/%m/%Y');
SELECT DATE_FORMAT(order_at, '%d-%b-%Y') FROM orders;
```

Use native dates for storage and logic; format only at presentation boundaries.

CAST and CONVERT

```

                                SQL
SELECT CAST('123.45' AS DECIMAL(10,2));
SELECT CAST(order_at AS DATE) FROM orders;
SELECT CONVERT(customer_name USING utf8mb4) FROM customers;
```

Implicit conversions can produce wrong results and prevent index use. Align joined column types and cast consciously.

COALESCE, IFNULL, NULLIF, and IF

```

                                SQL
SELECT
    COALESCE(shipping_city, c.city, 'Unknown') AS effective_city,
    IFNULL(discount_amount, 0) AS discount_amount,
    NULLIF(quantity, 0) AS nonzero_quantity,
    IF(status = 'delivered', 1, 0) AS delivered_flag
FROM orders AS o
JOIN customers AS c ON c.customer_id = o.customer_id
JOIN order_items AS oi ON oi.order_id = o.order_id;
```

COALESCE and CASE are portable standard SQL. IFNULL and IF are MySQL-specific conveniences.

9. Aggregate Functions, GROUP BY, and HAVING

Aggregation converts many input rows into summary values. The most common BA mistake is using the wrong aggregate or grouping at the wrong grain.

Core Aggregate Functions

Function	Meaning	NULL behavior
COUNT()	number of rows	counts every row
COUNT(column)	number of non-NULL values	excludes NULL
COUNT(DISTINCT column)	number of distinct non-NULL values	excludes NULL
SUM(column)	total	ignores NULL
AVG(column)	arithmetic mean	ignores NULL
MIN(column)	minimum	ignores NULL
MAX(column)	maximum	ignores NULL

```
SELECT
  COUNT(*) AS employee_count,
  SUM(years_employed) AS total_employee_years,
  AVG(years_employed) AS average_years,
  MAX(years_employed) AS longest_tenure
FROM employees;
```

SQL

COUNT(years_employed) answers "how many employees have a known years value?" It does not answer "total years worked." The word total years signals SUM(years_employed).

GROUP BY

```
SELECT
  shipping_city,
  COUNT(*) AS order_count
FROM orders
GROUP BY shipping_city
ORDER BY order_count DESC;
```

SQL

One output row is produced per distinct grouping combination. Every selected expression should be either a grouping expression or an aggregate. MySQL's ONLY_FULL_GROUP_BY mode rejects ambiguous non-grouped columns and should remain enabled.

```
SELECT
  DATE(order_at) AS order_date,
  status,
  COUNT(*) AS orders
FROM orders
GROUP BY DATE(order_at), status;
```

SQL

The output grain is one row per date-status pair.

WHERE versus HAVING

WHERE filters individual input rows before aggregation. HAVING filters groups after aggregation.

SQL

```
SELECT customer_id, COUNT(*) AS delivered_orders
FROM orders
WHERE status = 'delivered'           -- filter rows first
GROUP BY customer_id
HAVING COUNT(*) >= 5                 -- filter grouped results
ORDER BY delivered_orders DESC;
```

Do not use HAVING for ordinary row filters merely because MySQL permits some forms; early filtering is clearer and often more efficient.

COUNT DISTINCT

SQL

```
SELECT
    DATE(order_at) AS order_date,
    COUNT(*) AS orders,
    COUNT(DISTINCT customer_id) AS ordering_customers
FROM orders
GROUP BY DATE(order_at);
```

Distinct users are not additive across days: the same customer may appear on multiple days. Summing daily distinct counts does not equal monthly distinct customers.

Ratios and Percentages

General formula:

TEXT

```
rate = qualifying_count / eligible_count
percentage = 100 * qualifying_count / eligible_count
```

SQL implementation:

SQL

```
SELECT
    100.0 * SUM(status = 'cancelled') / NULLIF(COUNT(*), 0)
    AS cancellation_rate_pct
FROM orders;
```

In MySQL, boolean comparisons can evaluate to 1 or 0, but the portable form is:

SQL

```
100.0 * SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0)
```

Define the denominator carefully. Cancellation rate might mean cancelled orders divided by placed orders, paid orders, or orders old enough to reach a final state. Different denominators answer different questions.

Weighted Average

A simple average of item prices gives each row equal weight. An order-volume-weighted average is:

SQL

```
SELECT
    SUM(unit_price * quantity) / NULLIF(SUM(quantity), 0)
    AS weighted_average_selling_price
FROM order_items;
```

General formula:

```
weighted_average = SUM(value * weight) / SUM(weight)
```

TEXT

Avoiding Double Counting

Suppose one order has three items and two payment attempts. Joining orders -> order_items -> payments produces up to six rows for that order. Sums become inflated.

Wrong pattern:

```
SELECT SUM(oi.quantity * oi.unit_price), SUM(p.amount)
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN payments AS p ON p.order_id = o.order_id;
```

SQL

Correct pattern: aggregate each many-side to order grain before combining:

```
WITH item_totals AS (
  SELECT order_id,
         SUM(quantity * unit_price - discount_amount) AS order_value
  FROM order_items
  GROUP BY order_id
),
payment_totals AS (
  SELECT order_id,
         SUM(CASE WHEN payment_status = 'successful' THEN amount ELSE 0 END)
         AS successful_payment
  FROM payments
  GROUP BY order_id
)
SELECT
  SUM(i.order_value) AS merchandise_value,
  SUM(COALESCE(p.successful_payment, 0)) AS paid_value
FROM orders AS o
JOIN item_totals AS i ON i.order_id = o.order_id
LEFT JOIN payment_totals AS p ON p.order_id = o.order_id;
```

SQL

GROUP_CONCAT and WITH ROLLUP

```
SELECT category_id,
       GROUP_CONCAT(product_name ORDER BY product_name SEPARATOR ', ') AS products
FROM products
GROUP BY category_id;
```

SQL

GROUP_CONCAT has a configurable length limit. It is appropriate for display, not for creating normalized data.

```
SELECT shipping_city, status, COUNT(*) AS orders
FROM orders
GROUP BY shipping_city, status WITH ROLLUP;
```

SQL

WITH ROLLUP adds subtotal and grand-total rows represented using NULL-like grouping output. Label carefully, because genuine NULL groups can be confused with subtotal markers.

10. Joins: Combining Related Tables

A join combines rows based on a relationship. JOIN without a qualifier means INNER JOIN.

INNER JOIN

Returns only matching rows from both sides:

```
SELECT
    o.order_id,
    c.customer_name,
    o.order_at,
    o.status
FROM orders AS o
INNER JOIN customers AS c
    ON c.customer_id = o.customer_id;
```

SQL

For each order, SQL finds the customer whose key equals orders.customer_id. A customer with no order does not appear.

The attached SQLBolt-style exercise asked for sales for each movie. Selecting only sales values technically joins the tables but omits the identifying title. Include the entity identifier or name:

```
SELECT m.title, b.domestic_sales, b.international_sales
FROM movies AS m
JOIN boxoffice AS b ON b.movie_id = m.id;
```

SQL

LEFT JOIN

Returns every left-table row, plus matching right data. Unmatched right columns become NULL.

```
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS order_count
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name;
```

SQL

Use COUNT(o.order_id), not COUNT(), because the LEFT JOIN still produces one NULL-extended row for a customer with no orders. COUNT() would incorrectly count that as one.

RIGHT JOIN

Returns every right-table row plus matches from the left:

```
SELECT c.customer_name, o.order_id
FROM orders AS o
RIGHT JOIN customers AS c ON c.customer_id = o.customer_id;
```

SQL

Any RIGHT JOIN can be rewritten as a LEFT JOIN by swapping table order. Teams often standardize on LEFT JOIN for readability.

FULL OUTER JOIN

A full join returns matches plus unmatched rows from both tables. MySQL has no native FULL OUTER JOIN. A common emulation is:

```
SELECT a.id, a.value_a, b.value_b
FROM table_a AS a
LEFT JOIN table_b AS b ON b.id = a.id
UNION ALL
SELECT b.id, a.value_a, b.value_b
FROM table_b AS b
LEFT JOIN table_a AS a ON a.id = b.id
WHERE a.id IS NULL;
```

SQL

The second branch adds only right-side rows that lacked a left match.

CROSS JOIN

Returns the Cartesian product: every left row paired with every right row.

```
SELECT d.calendar_date, c.category_id
FROM calendar_dates AS d
CROSS JOIN categories AS c;
```

SQL

This is useful for building a date-category reporting spine. It is dangerous when accidental: 1 million rows crossed with 1 million rows theoretically creates 1 trillion combinations.

Self Join

A table can join to itself. An employee hierarchy example:

```
SELECT
    e.employee_name,
    m.employee_name AS manager_name
FROM employees AS e
LEFT JOIN employees AS m
    ON m.employee_id = e.manager_id;
```

SQL

Aliases distinguish the employee role from the manager role.

Non-Equi and Range Joins

Join conditions need not use equality:

```
SELECT p.product_id, p.list_price, b.band_name
FROM products AS p
JOIN price_bands AS b
    ON p.list_price >= b.min_price
    AND p.list_price < b.max_price;
```

SQL

Ensure ranges do not overlap unless multiple matches are intended.

ON versus WHERE with Outer Joins

These two queries are not equivalent.

Preserve all customers, matching only delivered orders:

SQL

```
SELECT c.customer_id, o.order_id
FROM customers AS c
LEFT JOIN orders AS o
  ON o.customer_id = c.customer_id
  AND o.status = 'delivered';
```

This version removes customers with no delivered order, effectively becoming an inner join:

SQL

```
SELECT c.customer_id, o.order_id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered';
```

A right-side condition in WHERE rejects the NULL-extended rows. Ask whether the condition defines matching right rows or filters the final result.

Semi-Join with EXISTS

Return customers who have at least one delivered order without multiplying customer rows:

SQL

```
SELECT c.customer_id, c.customer_name
FROM customers AS c
WHERE EXISTS (
  SELECT 1
  FROM orders AS o
  WHERE o.customer_id = c.customer_id
    AND o.status = 'delivered'
);
```

This often expresses existence more clearly than an inner join plus DISTINCT.

Anti-Join with NOT EXISTS

SQL

```
SELECT c.customer_id, c.customer_name
FROM customers AS c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders AS o
  WHERE o.customer_id = c.customer_id
);
```

Equivalent LEFT JOIN pattern:

SQL

```
SELECT c.customer_id, c.customer_name
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

Prefer NOT EXISTS when the intent is exclusion. Avoid NOT IN (subquery) if the subquery might contain NULL; one NULL can make all comparisons UNKNOWN.

Many-to-Many Joins

Orders and products relate through order_items:

SQL

```
SELECT
  o.order_id,
```

SQL

```
p.product_name,  
oi.quantity,  
oi.unit_price  
FROM orders AS o  
JOIN order_items AS oi ON oi.order_id = o.order_id  
JOIN products AS p ON p.product_id = oi.product_id;
```

The result is item grain, not order grain. State that before aggregating.

Diagnosing Join Multiplication

SQL

```
SELECT  
    o.order_id,  
    COUNT(*) AS joined_rows,  
    COUNT(DISTINCT oi.product_id) AS products,  
    COUNT(DISTINCT p.payment_id) AS payment_attempts  
FROM orders AS o  
LEFT JOIN order_items AS oi ON oi.order_id = o.order_id  
LEFT JOIN payments AS p ON p.order_id = o.order_id  
GROUP BY o.order_id  
HAVING COUNT(*) > GREATEST(COUNT(DISTINCT oi.product_id),  
                             COUNT(DISTINCT p.payment_id));
```

Before summing, compare COUNT() with expected key counts and inspect a single entity in detail.

11. Set Operations

Set operations stack compatible query results vertically. Joins combine columns horizontally.

UNION and UNION ALL

```
SELECT customer_id, order_at AS activity_at, 'order' AS activity_type
FROM orders
UNION ALL
SELECT customer_id, event_at, 'event'
FROM events;
```

SQL

Both branches must return the same number of columns in the same logical order with compatible types. Output column names come from the first query.

UNION removes duplicate result rows, which requires extra work. UNION ALL preserves all rows and is faster when deduplication is unnecessary.

```
SELECT city FROM customers
UNION
SELECT shipping_city FROM orders
ORDER BY city;
```

SQL

A final ORDER BY applies to the combined result.

INTERSECT and EXCEPT Portability

Some modern MySQL 8 releases support more set operators, but availability depends on the precise minor version and managed service. Portable alternatives are safer in interviews unless the environment is specified.

Intersection using EXISTS:

```
SELECT DISTINCT c.customer_id
FROM customers AS c
WHERE EXISTS (
    SELECT 1 FROM orders AS o WHERE o.customer_id = c.customer_id
);
```

SQL

Difference using NOT EXISTS:

```
SELECT c.customer_id
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1 FROM orders AS o WHERE o.customer_id = c.customer_id
);
```

SQL

12. Subqueries

A subquery is a query nested inside another SQL statement. Classify it by output shape and correlation.

Scalar Subquery

Returns one row and one column:

```
SELECT product_id, product_name, list_price
FROM products
WHERE list_price > (SELECT AVG(list_price) FROM products);
```

SQL

If the scalar subquery returns multiple rows, MySQL raises an error.

Single-Column Multi-Row Subquery

```
SELECT product_id, product_name
FROM products
WHERE category_id IN (
    SELECT category_id
    FROM categories
    WHERE category_name IN ('Fashion', 'Home')
);
```

SQL

Subquery in SELECT

```
SELECT
    c.customer_id,
    c.customer_name,
    (SELECT COUNT(*)
     FROM orders AS o
     WHERE o.customer_id = c.customer_id) AS order_count
FROM customers AS c;
```

SQL

This correlated scalar subquery is clear for small workloads, but pre-aggregation plus a join may scale better.

Derived Table in FROM

```
SELECT monthly.order_month, monthly.revenue
FROM (
    SELECT DATE_FORMAT(o.order_at, '%Y-%m') AS order_month,
           SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS revenue
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.order_id
    WHERE o.status = 'delivered'
    GROUP BY DATE_FORMAT(o.order_at, '%Y-%m')
) AS monthly
WHERE monthly.revenue > 1000000;
```

SQL

A derived table requires an alias in MySQL.

Correlated Subquery

A correlated subquery references the current outer row:

```
SELECT p.product_id, p.product_name, p.list_price
FROM products AS p
WHERE p.list_price > (
    SELECT AVG(p2.list_price)
    FROM products AS p2
    WHERE p2.category_id = p.category_id
);
```

SQL

This returns products priced above their own category average.

ANY, SOME, and ALL

```
-- Greater than at least one comparison value
WHERE list_price > ANY (
    SELECT list_price FROM products WHERE category_id = 5
)

-- Greater than every comparison value
WHERE list_price > ALL (
    SELECT list_price FROM products WHERE category_id = 5
)
```

SQL

SOME is a synonym for ANY. Empty sets and NULLs require careful reasoning. Often MIN or MAX expresses the intent more clearly.

Comparing One Row's Value to the Same Column

To find movies longer than WALL-E:

```
SELECT title, length_minutes
FROM movies
WHERE length_minutes > (
    SELECT length_minutes
    FROM movies
    WHERE title = 'WALL-E'
);
```

SQL

The inner query obtains one reference value; the outer query compares every row against it. Ensure the inner condition uniquely identifies one row.

When to Use Each Form

Need	Usually choose
Compare with one aggregate	scalar subquery or CTE
Test existence	EXISTS / NOT EXISTS
Filter against a small set	IN subquery
Reuse a multi-step result	CTE
Compare each row with its group	window function or correlated subquery
Return columns from both tables	JOIN

13. Common Table Expressions (CTEs)

A CTE names a query result for use within one statement. It improves decomposition and readability; it is not automatically a cached result.

Basic CTE

SQL

```
WITH delivered_order_values AS (
  SELECT
    o.order_id,
    o.customer_id,
    o.order_at,
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS order_value
  FROM orders AS o
  JOIN order_items AS oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY o.order_id, o.customer_id, o.order_at
)
SELECT
  customer_id,
  COUNT(*) AS delivered_orders,
  SUM(order_value) AS customer_revenue
FROM delivered_order_values
GROUP BY customer_id;
```

Multiple CTEs

SQL

```
WITH order_values AS (
  SELECT order_id,
    SUM(quantity * unit_price - discount_amount) AS order_value
  FROM order_items
  GROUP BY order_id
),
customer_metrics AS (
  SELECT o.customer_id,
    COUNT(*) AS orders,
    SUM(v.order_value) AS revenue
  FROM orders AS o
  JOIN order_values AS v ON v.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY o.customer_id
)
SELECT *
FROM customer_metrics
WHERE orders >= 3
ORDER BY revenue DESC;
```

Give each CTE one clear responsibility and verify it independently during development.

CTE versus Alternatives

Object	Scope	Reusable across statements?	Typical use
Derived table	one query block	no	compact one-off step
CTE	one statement	no	readable multi-step logic

Temporary table	session	yes	reuse, indexing, debugging
View	database object	yes	shared abstraction/security
Summary table	persisted	yes	precomputed analytics

The MySQL optimizer may merge or materialize CTEs. Inspect EXPLAIN; do not assume either behavior.

Recursive CTE

A recursive CTE has an anchor query and a recursive query, connected with UNION ALL:

```
WITH RECURSIVE numbers AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1
  FROM numbers
  WHERE n < 10
)
SELECT n FROM numbers;
```

SQL

Generate a date series:

```
WITH RECURSIVE calendar AS (
  SELECT DATE('2026-01-01') AS calendar_date
  UNION ALL
  SELECT calendar_date + INTERVAL 1 DAY
  FROM calendar
  WHERE calendar_date < '2026-01-31'
)
SELECT calendar_date FROM calendar;
```

SQL

Employee hierarchy:

```
WITH RECURSIVE org AS (
  SELECT employee_id, employee_name, manager_id,
         0 AS depth,
         CAST(employee_name AS CHAR(1000)) AS path
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  SELECT e.employee_id, e.employee_name, e.manager_id,
         o.depth + 1,
         CONCAT(o.path, ' > ', e.employee_name)
  FROM employees AS e
  JOIN org AS o ON e.manager_id = o.employee_id
  WHERE o.depth < 20
)
SELECT * FROM org ORDER BY path;
```

SQL

Always include a termination condition. Guard against cycles in dirty hierarchy data, and know the server's recursion-depth setting.

14. Window Functions

Window functions calculate across related rows without collapsing them. Aggregation with GROUP BY returns one row per group; a window function preserves the original row grain.

Window Syntax

```
function(expression) OVER (
  PARTITION BY grouping_columns
  ORDER BY ordering_columns
  ROWS BETWEEN frame_start AND frame_end
)
```

SQL

PARTITION BY restarts the calculation for each group. ORDER BY establishes sequence. The frame specifies which rows around the current row participate.

Aggregate versus Window

```
-- One row per customer
SELECT customer_id, SUM(amount) AS total_amount
FROM payments
GROUP BY customer_id;

-- One row per payment, with customer total repeated
SELECT payment_id, customer_id, amount,
       SUM(amount) OVER (PARTITION BY customer_id) AS customer_total
FROM payments;
```

SQL

ROW_NUMBER, RANK, and DENSE_RANK

```
SELECT
  product_id,
  category_id,
  list_price,
  ROW_NUMBER() OVER (
    PARTITION BY category_id
    ORDER BY list_price DESC, product_id
  ) AS row_num,
  RANK() OVER (
    PARTITION BY category_id
    ORDER BY list_price DESC
  ) AS price_rank,
  DENSE_RANK() OVER (
    PARTITION BY category_id
    ORDER BY list_price DESC
  ) AS dense_price_rank
FROM products;
```

SQL

For prices 100, 100, 80:

Function	Ranks
ROW_NUMBER	1, 2, 3; arbitrary among ties unless tie-breaker exists
RANK	1, 1, 3; leaves a gap

DENSE_RANK | 1, 1, 2; no gap

Clarify whether "top three" means exactly three rows or all products in the top three distinct ranks.

Top N per Group

MySQL has no QUALIFY, so filter window results in an outer query or CTE:

```
WITH ranked AS (
  SELECT
    p.*,
    ROW_NUMBER() OVER (
      PARTITION BY category_id
      ORDER BY list_price DESC, product_id
    ) AS rn
  FROM products AS p
)
SELECT *
FROM ranked
WHERE rn <= 3;
```

SQL

LAG and LEAD

```
WITH monthly AS (
  SELECT
    DATE_FORMAT(o.order_at, '%Y-%m-01') AS month_start,
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS revenue
  FROM orders AS o
  JOIN order_items AS oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY DATE_FORMAT(o.order_at, '%Y-%m-01')
)
SELECT
  month_start,
  revenue,
  LAG(revenue) OVER (ORDER BY month_start) AS previous_revenue,
  revenue - LAG(revenue) OVER (ORDER BY month_start) AS absolute_change,
  100.0 * (revenue - LAG(revenue) OVER (ORDER BY month_start))
    / NULLIF(LAG(revenue) OVER (ORDER BY month_start), 0)
    AS growth_pct
FROM monthly;
```

SQL

LEAD accesses a later row. Both are valuable for status transitions, gaps between events, and period comparisons.

Running Total

```
SELECT
  order_date,
  daily_revenue,
  SUM(daily_revenue) OVER (
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_revenue
FROM daily_sales;
```

SQL

Specify ROWS explicitly. With duplicate ordering values, the default RANGE frame can include peers and cause jumps.

Moving Average

SQL

```
SELECT
  order_date,
  daily_orders,
  AVG(daily_orders) OVER (
    ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS seven_row_moving_average
FROM daily_metrics;
```

This is a seven-row average, which equals seven calendar days only when every date is present. Join to a calendar spine first when missing dates matter.

FIRST_VALUE, LAST_VALUE, and NTH_VALUE

SQL

```
SELECT
  customer_id,
  order_id,
  order_at,
  FIRST_VALUE(order_id) OVER (
    PARTITION BY customer_id
    ORDER BY order_at, order_id
  ) AS first_order_id,
  LAST_VALUE(order_id) OVER (
    PARTITION BY customer_id
    ORDER BY order_at, order_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS last_order_id
FROM orders;
```

LAST_VALUE often surprises learners because the default ordered frame ends at the current row. Extend the frame to UNBOUNDED FOLLOWING for the partition's final value.

NTILE, PERCENT_RANK, and CUME_DIST

SQL

```
SELECT
  customer_id,
  customer_revenue,
  NTILE(4) OVER (ORDER BY customer_revenue DESC) AS revenue_quartile,
  PERCENT_RANK() OVER (ORDER BY customer_revenue) AS percent_rank,
  CUME_DIST() OVER (ORDER BY customer_revenue) AS cumulative_distribution
FROM customer_metrics;
```

NTILE(4) divides rows into four nearly equal-sized buckets; tied values can fall into different buckets. Use business thresholds when ties must remain together.

Named Windows

SQL

```
SELECT
  order_id,
  customer_id,
  order_at,
  ROW_NUMBER() OVER w AS order_sequence,
  LAG(order_at) OVER w AS previous_order_at
FROM orders
WINDOW w AS (PARTITION BY customer_id ORDER BY order_at, order_id);
```

Deduplicate to Latest Row

SQL

```
WITH latest AS (  
  SELECT  
    s.*,  
    ROW_NUMBER() OVER (  
      PARTITION BY order_id  
      ORDER BY status_at DESC, status_history_id DESC  
    ) AS rn  
  FROM order_status_history AS s  
)  
SELECT * FROM latest WHERE rn = 1;
```

A deterministic tie-breaker is essential when timestamps can be equal.

15. Business Analytics SQL Patterns

This chapter turns SQL syntax into the metric ownership, root-cause diagnosis, and reporting work expected from a Business Analyst.

Metric Definition Template

Before querying, document:

1. Business question: What decision will this metric support?
2. Entity/grain: one row per day, customer, order, seller, or category?
3. Numerator and denominator: exactly which events or entities?
4. Inclusions/exclusions: cancelled, returned, test, fraudulent, or incomplete records?
5. Time field: event time, order time, payment time, delivery time, or processing time?
6. Time zone and period: calendar day in which business zone?
7. Late-arriving data: when is a period considered complete?
8. Dimensions: which segment definitions and historical version?

A technically correct query can still answer the wrong business question if any of these are unclear.

Revenue and Order Value

Item-level gross merchandise value:

```
SELECT
    SUM(quantity * unit_price) AS gross_item_value,
    SUM(discount_amount) AS discounts,
    SUM(quantity * unit_price - discount_amount) AS net_item_value
FROM order_items;
```

SQL

Delivered revenue by month:

```
SELECT
    DATE_FORMAT(o.order_at, '%Y-%m-01') AS month_start,
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS delivered_value
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
WHERE o.status = 'delivered'
GROUP BY DATE_FORMAT(o.order_at, '%Y-%m-01')
ORDER BY month_start;
```

SQL

Do not label a metric "revenue" until the business confirms whether it is booked, paid, delivered, net of returns, net of tax, or recognized under accounting policy.

Average Order Value

```
AOV = total eligible order value / number of eligible orders
```

TEXT

Compute order values first so each order receives equal weight:

SQL

```

WITH order_values AS (
  SELECT o.order_id,
         SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS order_value
  FROM orders AS o
  JOIN order_items AS oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
  GROUP BY o.order_id
)
SELECT
  SUM(order_value) AS total_value,
  COUNT(*) AS orders,
  SUM(order_value) / NULLIF(COUNT(*), 0) AS average_order_value
FROM order_values;

```

AVG(unit_price) is not AOV.

Conversion Rate and Funnel

Suppose events contain view_product, add_to_cart, checkout, and purchase. Define whether conversion is user-based, session-based, or event-based.

Session funnel with one row per session:

SQL

```

WITH session_flags AS (
  SELECT
    session_id,
    MAX(event_name = 'view_product') AS viewed,
    MAX(event_name = 'add_to_cart') AS added,
    MAX(event_name = 'checkout') AS checked_out,
    MAX(event_name = 'purchase') AS purchased
  FROM events
  WHERE event_at >= '2026-07-01'
    AND event_at < '2026-08-01'
  GROUP BY session_id
)
SELECT
  COUNT(*) AS sessions,
  SUM(viewed) AS view_sessions,
  SUM(added) AS cart_sessions,
  SUM(checked_out) AS checkout_sessions,
  SUM(purchased) AS purchase_sessions,
  100.0 * SUM(added) / NULLIF(SUM(viewed), 0) AS view_to_cart_pct,
  100.0 * SUM(purchased) / NULLIF(SUM(viewed), 0) AS view_to_purchase_pct
FROM session_flags;

```

This is a non-sequential funnel: a session merely contains each event. A strict sequential funnel must check event order and timestamps.

Root-Cause Segmentation

When a KPI drops, first verify the definition and data pipeline, then segment systematically:

SQL

```

WITH daily_city AS (
  SELECT DATE(order_at) AS dt,
         shipping_city,
         COUNT(*) AS orders,
         SUM(status = 'cancelled') AS cancelled
  FROM orders
  WHERE order_at >= '2026-07-01'
    AND order_at < '2026-07-15'

```

SQL

```

    GROUP BY DATE(order_at), shipping_city
)
SELECT
    shipping_city,
    SUM(CASE WHEN dt < '2026-07-08' THEN orders ELSE 0 END) AS before_orders,
    SUM(CASE WHEN dt >= '2026-07-08' THEN orders ELSE 0 END) AS after_orders,
    100.0 * (
        SUM(CASE WHEN dt >= '2026-07-08' THEN orders ELSE 0 END)
        - SUM(CASE WHEN dt < '2026-07-08' THEN orders ELSE 0 END)
    ) / NULLIF(SUM(CASE WHEN dt < '2026-07-08' THEN orders ELSE 0 END), 0)
    AS change_pct
FROM daily_city
GROUP BY shipping_city
ORDER BY change_pct;

```

Compare equal-length periods, account for weekday/seasonality, and avoid claiming causality from a segment difference alone.

Period-over-Period Growth

TEXT

```
growth_rate = (current - previous) / previous
```

SQL

```

WITH monthly AS (
    SELECT DATE_FORMAT(order_at, '%Y-%m-01') AS month_start,
           COUNT(*) AS orders
    FROM orders
    GROUP BY DATE_FORMAT(order_at, '%Y-%m-01')
), compared AS (
    SELECT month_start, orders,
           LAG(orders) OVER (ORDER BY month_start) AS previous_orders
    FROM monthly
)
SELECT *,
       orders - previous_orders AS absolute_change,
       100.0 * (orders - previous_orders) / NULLIF(previous_orders, 0)
       AS growth_pct
FROM compared;

```

A missing month causes LAG to compare non-adjacent periods. Join metrics to a calendar month spine when continuity matters.

Share of Total

SQL

```

WITH category_sales AS (
    SELECT p.category_id,
           SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS sales
    FROM order_items AS oi
    JOIN products AS p ON p.product_id = oi.product_id
    GROUP BY p.category_id
)
SELECT
    category_id,
    sales,
    100.0 * sales / NULLIF(SUM(sales) OVER (), 0) AS share_pct
FROM category_sales
ORDER BY sales DESC;

```

Pareto / 80-20 Analysis

SQL

```
WITH product_sales AS (
    SELECT product_id,
           SUM(quantity * unit_price - discount_amount) AS sales
    FROM order_items
    GROUP BY product_id
), cumulative AS (
    SELECT product_id, sales,
           SUM(sales) OVER (ORDER BY sales DESC, product_id) AS running_sales,
           SUM(sales) OVER () AS total_sales
    FROM product_sales
)
SELECT *, 100.0 * running_sales / NULLIF(total_sales, 0) AS cumulative_share_pct
FROM cumulative
ORDER BY sales DESC, product_id;
```

Find where cumulative share first reaches 80%; do not assume exactly 20% of products produce 80% of value.

Cohort Retention

Define cohort as the month of first delivered purchase and activity as a delivered purchase in a later month.

SQL

```
WITH customer_orders AS (
    SELECT DISTINCT
           customer_id,
           STR_TO_DATE(DATE_FORMAT(order_at, '%Y-%m-01'), '%Y-%m-%d') AS activity_month
    FROM orders
    WHERE status = 'delivered'
), cohort AS (
    SELECT customer_id, MIN(activity_month) AS cohort_month
    FROM customer_orders
    GROUP BY customer_id
), activity AS (
    SELECT
           c.cohort_month,
           o.activity_month,
           TIMESTAMPTDIFF(MONTH, c.cohort_month, o.activity_month) AS month_number,
           COUNT(DISTINCT o.customer_id) AS active_customers
    FROM customer_orders AS o
    JOIN cohort AS c ON c.customer_id = o.customer_id
    GROUP BY c.cohort_month, o.activity_month,
           TIMESTAMPTDIFF(MONTH, c.cohort_month, o.activity_month)
), sized AS (
    SELECT *,
           MAX(CASE WHEN month_number = 0 THEN active_customers END)
           OVER (PARTITION BY cohort_month) AS cohort_size
    FROM activity
)
SELECT
    cohort_month,
    month_number,
    active_customers,
    cohort_size,
    100.0 * active_customers / NULLIF(cohort_size, 0) AS retention_pct
FROM sized
ORDER BY cohort_month, month_number;
```

Retention definitions vary: any activity, purchase, paid purchase, or delivered order. State the choice.

Repeat Purchase and Reactivation

SQL

```
WITH sequenced AS (
  SELECT
    customer_id,
    order_id,
    order_at,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id ORDER BY order_at, order_id
    ) AS order_number,
    LAG(order_at) OVER (
      PARTITION BY customer_id ORDER BY order_at, order_id
    ) AS previous_order_at
  FROM orders
  WHERE status = 'delivered'
)
SELECT
  COUNT(DISTINCT CASE WHEN order_number = 1 THEN customer_id END) AS first_time_customers,
  COUNT(DISTINCT CASE WHEN order_number >= 2 THEN customer_id END) AS repeat_customers,
  COUNT(DISTINCT CASE
    WHEN TIMESTAMPDIFF(DAY, previous_order_at, order_at) >= 90
    THEN customer_id END) AS reactivated_customers
FROM sequenced;
```

The 90-day reactivation threshold is a business assumption, not a universal rule.

RFM Segmentation

RFM means recency, frequency, and monetary value.

SQL

```
WITH customer_rfm AS (
  SELECT
    o.customer_id,
    DATEDIFF('2026-07-18', MAX DATE(o.order_at)) AS recency_days,
    COUNT(DISTINCT o.order_id) AS frequency,
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS monetary
  FROM orders AS o
  JOIN order_items AS oi ON oi.order_id = o.order_id
  WHERE o.status = 'delivered'
    AND o.order_at < '2026-07-19'
  GROUP BY o.customer_id
), scored AS (
  SELECT *,
    6 - NTILE(5) OVER (ORDER BY recency_days ASC) AS recency_score,
    NTILE(5) OVER (ORDER BY frequency ASC) AS frequency_score,
    NTILE(5) OVER (ORDER BY monetary ASC) AS monetary_score
  FROM customer_rfm
)
SELECT *, CONCAT(recency_score, frequency_score, monetary_score) AS rfm_code
FROM scored;
```

Review scoring direction carefully: lower recency days is better, whereas higher frequency and monetary are better.

Sessionization

When no session ID exists, define a new session after 30 minutes of inactivity:

SQL

```

WITH with_previous AS (
    SELECT
        customer_id,
        event_at,
        event_name,
        LAG(event_at) OVER (
            PARTITION BY customer_id ORDER BY event_at, event_id
        ) AS previous_event_at
    FROM events
    WHERE customer_id IS NOT NULL
), boundaries AS (
    SELECT *,
        CASE WHEN previous_event_at IS NULL
            OR TIMESTAMPDIFF(MINUTE, previous_event_at, event_at) >= 30
            THEN 1 ELSE 0 END AS new_session
    FROM with_previous
), assigned AS (
    SELECT *,
        SUM(new_session) OVER (
            PARTITION BY customer_id
            ORDER BY event_at
            ROWS UNBOUNDED PRECEDING
        ) AS session_number
    FROM boundaries
)
SELECT customer_id, session_number,
    MIN(event_at) AS session_start,
    MAX(event_at) AS session_end,
    COUNT(*) AS events
FROM assigned
GROUP BY customer_id, session_number;

```

Gaps and Islands

Identify consecutive active dates by subtracting row number from each date:

SQL

```

WITH active_dates AS (
    SELECT DISTINCT customer_id, DATE(event_at) AS activity_date
    FROM events
), numbered AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id ORDER BY activity_date
        ) AS rn
    FROM active_dates
), grouped AS (
    SELECT *, DATE_SUB(activity_date, INTERVAL rn DAY) AS island_key
    FROM numbered
)
SELECT customer_id,
    MIN(activity_date) AS streak_start,
    MAX(activity_date) AS streak_end,
    COUNT(*) AS streak_days
FROM grouped
GROUP BY customer_id, island_key;

```

Pivot and Unpivot

Pivot using conditional aggregation:

SQL

```
SELECT
    shipping_city,
    SUM(CASE WHEN status = 'delivered' THEN 1 ELSE 0 END) AS delivered,
    SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS cancelled,
    SUM(CASE WHEN status = 'returned' THEN 1 ELSE 0 END) AS returned
FROM orders
GROUP BY shipping_city;
```

Unpivot fixed columns with UNION ALL:

SQL

```
SELECT order_id, 'subtotal' AS measure, subtotal AS value FROM order_summary
UNION ALL
SELECT order_id, 'shipping', shipping_fee FROM order_summary
UNION ALL
SELECT order_id, 'tax', tax_amount FROM order_summary;
```

Date Spine and Zero-Filling

Facts cannot produce a row for a day with no facts. Build or maintain a calendar table, then LEFT JOIN:

SQL

```
SELECT
    c.calendar_date,
    COALESCE(COUNT(o.order_id), 0) AS orders
FROM calendar AS c
LEFT JOIN orders AS o
    ON o.order_at >= c.calendar_date
    AND o.order_at < c.calendar_date + INTERVAL 1 DAY
WHERE c.calendar_date >= '2026-07-01'
    AND c.calendar_date < '2026-08-01'
GROUP BY c.calendar_date
ORDER BY c.calendar_date;
```

16. INSERT, UPDATE, DELETE, and Safe Data Changes

INSERT

```
INSERT INTO categories (category_name)
VALUES ('Beauty');

INSERT INTO categories (category_name)
VALUES ('Books'), ('Sports'), ('Grocery');
```

SQL

Always list columns. Relying on physical column order makes code fragile.

Copy from a query:

```
INSERT INTO archived_orders (order_id, customer_id, order_at, status)
SELECT order_id, customer_id, order_at, status
FROM orders
WHERE order_at < '2024-01-01';
```

SQL

Upsert

```
INSERT INTO daily_metrics (metric_date, order_count, revenue)
VALUES ('2026-07-18', 1500, 2450000.00) AS incoming
ON DUPLICATE KEY UPDATE
    order_count = incoming.order_count,
    revenue = incoming.revenue;
```

SQL

Syntax details vary across MySQL versions. Upsert is triggered by a primary or unique-key conflict. Ensure the unique key represents the business identity of the row.

REPLACE deletes the conflicting row and inserts another; it can change auto-increment IDs, trigger cascades, and reset unspecified columns. Prefer explicit upsert semantics.

UPDATE

```
UPDATE products
SET list_price = ROUND(list_price * 1.05, 2),
    updated_at = CURRENT_TIMESTAMP
WHERE category_id = 8
    AND active_flag = TRUE;
```

SQL

Update using another table:

```
UPDATE products AS p
JOIN supplier_price_changes AS s ON s.product_id = p.product_id
SET p.unit_cost = s.new_unit_cost
WHERE s.effective_date <= CURRENT_DATE();
```

SQL

Safe workflow:

```
START TRANSACTION;
```

SQL

SQL

```
SELECT product_id, list_price
FROM products
WHERE category_id = 8 AND active_flag = TRUE
FOR UPDATE;

UPDATE products
SET list_price = ROUND(list_price * 1.05, 2)
WHERE category_id = 8 AND active_flag = TRUE;

SELECT ROW_COUNT() AS changed_rows;
-- Validate samples and totals.
COMMIT; -- or ROLLBACK
```

DELETE

SQL

```
DELETE FROM events
WHERE event_at < '2024-01-01';
```

Delete with a join:

SQL

```
DELETE e
FROM events AS e
JOIN blocked_customers AS b ON b.customer_id = e.customer_id
WHERE e.event_at < '2026-01-01';
```

For large deletes, use controlled batches and monitor locks, replication, and transaction logs. Retention and deletion of personal data must follow policy and law.

DELETE versus TRUNCATE versus DROP

Command	Removes rows	Keeps structure	WHERE allowed	Typical transaction behavior
DELETE	selected or all	yes	yes	DML; can roll back in transaction
TRUNCATE	all	yes	no	DDL; implicit commit behavior
DROP	object and data	no	no	DDL; implicit commit behavior

17. Table Design, Constraints, and Normalization

CREATE TABLE Anatomy

```
CREATE TABLE sellers (
  seller_id      BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  seller_code    VARCHAR(30) NOT NULL UNIQUE,
  seller_name    VARCHAR(120) NOT NULL,
  joined_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status         VARCHAR(20) NOT NULL DEFAULT 'active',
  rating         DECIMAL(3,2),
  created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT chk_seller_rating CHECK (rating BETWEEN 0 AND 5),
  INDEX idx_sellers_status_joined (status, joined_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

SQL

Constraints

- NOT NULL: value is required.
- DEFAULT: value used when omitted; it does not replace explicit NULL unless rules say so.
- PRIMARY KEY: unique, non-NULL identity.
- UNIQUE: prevents duplicate key combinations; NULL behavior can allow multiple NULLs.
- FOREIGN KEY: enforces referenced parent existence.
- CHECK: enforces a boolean condition in modern MySQL 8.

Foreign-key actions:

```
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
  ON UPDATE RESTRICT
  ON DELETE RESTRICT
```

SQL

CASCADE propagates parent changes/deletes. SET NULL preserves the child but requires a nullable column. Choose actions from business lifecycle rules, not convenience.

Generated Columns

```
CREATE TABLE line_examples (
  quantity INT NOT NULL,
  unit_price DECIMAL(12,2) NOT NULL,
  line_value DECIMAL(14,2)
  GENERATED ALWAYS AS (quantity * unit_price) STORED
);
```

SQL

Generated columns can centralize deterministic expressions and may be indexed. Do not duplicate fast-changing business logic thoughtlessly.

Temporary Tables

SQL

```
CREATE TEMPORARY TABLE high_value_customers AS
SELECT customer_id, SUM(order_value) AS revenue
FROM order_level_values
GROUP BY customer_id
HAVING SUM(order_value) >= 100000;

CREATE INDEX idx_hvc_revenue ON high_value_customers(revenue);
```

Temporary tables last for the session, can be indexed, and help debug or reuse expensive intermediate data. They consume resources and introduce state, so clean them up when appropriate.

Normalization

Normalization reduces redundancy and anomalies.

First Normal Form (1NF): each column holds atomic values, no repeating groups. Do not store `product_ids = '4,7,9'`; use `order_items`.

Second Normal Form (2NF): be in 1NF, and every non-key attribute depends on the entire candidate key. In `(order_id, product_id)`, `product_name` depends only on `product_id` and belongs in `products`.

Third Normal Form (3NF): be in 2NF, and non-key attributes do not depend transitively on another non-key attribute. If `postal_code -> city`, repeating city with each customer may introduce inconsistencies.

BCNF awareness: every determinant should be a candidate key; it handles certain edge cases beyond 3NF.

Normalization prevents:

- Update anomaly: changing the same fact in many rows.
- Insert anomaly: inability to record one fact without another.
- Delete anomaly: deleting one fact accidentally removes another.

Denormalization

Analytics systems may duplicate or precompute data to reduce query cost. Denormalization trades simpler/faster reads for storage, refresh complexity, and possible inconsistency. Every duplicated fact needs an owner and reconciliation method.

OLTP versus OLAP

Characteristic	OLTP	OLAP / warehouse
Workload	many small writes and point reads	scans, joins, aggregations
Model	normalized	dimensional/denormalized
Data	current operational state	integrated history
Users	application services	analysts and BI tools

Star Schema

A fact table contains measurable events at a declared grain. Dimension tables contain descriptive context.

Example sales star:

TEXT

```
fact_order_items: one row per delivered order item
  date_key, customer_key, product_key, seller_key, order_id
  quantity, gross_value, discount_value, net_value, cost

dim_date, dim_customer, dim_product, dim_seller
```

Measures may be:

- Additive: sales can be summed across all dimensions.
- Semi-additive: account balance can be summed across accounts but not across dates.
- Non-additive: rates and ratios should be recomputed from components.

Slowly changing dimension Type 1 overwrites attributes; Type 2 creates a new version with effective dates, preserving historical truth.

18. Transactions, ACID, and Concurrency

ACID

- Atomicity: all steps succeed or none do.
- Consistency: constraints and business invariants remain valid.
- Isolation: concurrent transactions behave according to a defined isolation level.
- Durability: committed work survives failures according to the system's guarantees.

Money transfer illustrates atomicity: debit and credit must commit together.

Transaction Commands

SQL

```
SET autocommit = 0;
START TRANSACTION;

UPDATE inventory
SET available_units = available_units - 2
WHERE product_id = 1001 AND available_units >= 2;

SAVEPOINT inventory_reserved;

INSERT INTO orders (customer_id, order_at, status)
VALUES (501, NOW(), 'placed');

-- If only the later work fails:
ROLLBACK TO SAVEPOINT inventory_reserved;
-- Otherwise COMMIT or full ROLLBACK.

COMMIT;
SET autocommit = 1;
```

Keep transactions short. Never wait for human input while holding locks.

Isolation Levels and Anomalies

Isolation level	Dirty reads	Non-repeatable reads	Phantom risk / behavior
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	prevented	possible	possible
REPEATABLE READ	prevented	prevented for snapshot reads	InnoDB uses MVCC and locking behavior
SERIALIZABLE	prevented	prevented	strongest, lowest concurrency

MySQL InnoDB defaults to REPEATABLE READ. Exact behavior depends on consistent reads versus locking reads.

SQL

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
```

Anomalies:

- Dirty read: see another transaction's uncommitted change.

- Non-repeatable read: rereading one row produces a changed value.
- Phantom: rerunning a predicate returns a changed set of rows.
- Lost update: concurrent writers overwrite each other's changes.

MVCC and Locking Reads

Multi-Version Concurrency Control lets ordinary consistent reads view a snapshot without blocking many writers.

Use a locking read when you will make a decision and update based on current values:

```
START TRANSACTION;
SELECT available_units
FROM inventory
WHERE product_id = 1001
FOR UPDATE;

UPDATE inventory
SET available_units = available_units - 1
WHERE product_id = 1001;
COMMIT;
```

SQL

FOR SHARE obtains a shared locking read. InnoDB can use record, gap, and next-key locks depending on index and isolation level.

Deadlocks

A deadlock occurs when transactions wait cyclically. The database aborts one transaction. Applications should retry appropriate transactions.

Reduce deadlocks by:

- Accessing tables and rows in a consistent order.
- Keeping transactions short.
- Indexing predicates so fewer rows are locked.
- Avoiding unnecessary user interaction inside transactions.
- Handling retryable errors explicitly.

Optimistic Concurrency

```
UPDATE products
SET list_price = 1099.00,
    version_number = version_number + 1
WHERE product_id = 42
    AND version_number = 7;
```

SQL

If ROW_COUNT() is zero, another writer changed the row or it no longer exists. Reload and decide whether to retry.

19. Views, Stored Programs, Triggers, and Events

Views

A view stores a query definition, not ordinary materialized result data.

SQL

```
CREATE OR REPLACE VIEW delivered_order_summary AS
SELECT
    o.order_id,
    o.customer_id,
    o.order_at,
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS order_value
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
WHERE o.status = 'delivered'
GROUP BY o.order_id, o.customer_id, o.order_at;

SELECT * FROM delivered_order_summary
WHERE order_at >= '2026-07-01';

DROP VIEW delivered_order_summary;
```

View uses:

- Hide complex joins behind a stable interface.
- Restrict exposed columns or rows.
- Centralize a carefully governed definition.
- Support backward-compatible interfaces.

Risks: nested views can hide expensive work; changes can affect many consumers; and a view does not automatically improve performance. MySQL has no native general-purpose materialized view. Managed summary tables can emulate one but require refresh and reconciliation logic.

Some simple views are updatable. Aggregation, DISTINCT, GROUP BY, UNION, and many joins generally make a view non-updatable.

SQL

```
CREATE VIEW active_products AS
SELECT product_id, product_name, list_price, active_flag
FROM products
WHERE active_flag = TRUE
WITH CHECK OPTION;
```

WITH CHECK OPTION prevents changes through the view that would make rows disappear from its condition.

Stored Procedures

SQL

```
DELIMITER //

CREATE PROCEDURE GetCustomerOrders(
    IN p_customer_id BIGINT,
    IN p_start_date DATE,
    OUT p_order_count INT
```

SQL

```

)
BEGIN
    SELECT COUNT(*) INTO p_order_count
    FROM orders
    WHERE customer_id = p_customer_id
    AND order_at >= p_start_date;

    SELECT order_id, order_at, status
    FROM orders
    WHERE customer_id = p_customer_id
    AND order_at >= p_start_date
    ORDER BY order_at;
END //

DELIMITER ;

CALL GetCustomerOrders(501, '2026-01-01', @count);
SELECT @count;

```

DELIMITER is a client instruction that temporarily changes how statement endings are detected; it is not part of the stored procedure itself.

Parameter modes:

- IN: input only.
- OUT: value returned through parameter.
- INOUT: caller supplies and receives a value.

Procedures can use variables, IF, CASE, loops, cursors, and exception handlers. Prefer set-based SQL over row-by-row cursors when possible.

SQL

```

DECLARE v_done BOOLEAN DEFAULT FALSE;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

```

Stored Functions

SQL

```

DELIMITER //
CREATE FUNCTION NetLineValue(
    p_quantity INT,
    p_unit_price DECIMAL(12,2),
    p_discount DECIMAL(12,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
RETURN p_quantity * p_unit_price - COALESCE(p_discount, 0) //
DELIMITER ;

```

A function returns one value and can be called in expressions. Avoid hiding expensive data access or frequently changing business rules in functions used across millions of rows.

Triggers

SQL

```

DELIMITER //
CREATE TRIGGER products_before_update
BEFORE UPDATE ON products
FOR EACH ROW
BEGIN
    IF NEW.list_price < 0 THEN
        SIGNAL SQLSTATE '45000'

```

SQL

```

        SET MESSAGE_TEXT = 'list_price cannot be negative';
    END IF;
END //
DELIMITER ;

```

Trigger timing/events are BEFORE or AFTER with INSERT, UPDATE, or DELETE. Refer to NEW.column and, where applicable, OLD.column.

Triggers can enforce local invariants or write audit records, but they create hidden side effects, complicate bulk loads, and can surprise applications. Prefer declarative constraints where they can express the rule.

Events

SQL

```

SET GLOBAL event_scheduler = ON;

CREATE EVENT refresh_daily_summary
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_DATE + INTERVAL 1 DAY
DO
    INSERT INTO daily_metrics (metric_date, order_count)
    SELECT CURRENT_DATE - INTERVAL 1 DAY, COUNT(*)
    FROM orders
    WHERE order_at >= CURRENT_DATE - INTERVAL 1 DAY
        AND order_at < CURRENT_DATE
    ON DUPLICATE KEY UPDATE order_count = VALUES(order_count);

```

Database events require privileges, monitoring, idempotency, and failure handling. External workflow schedulers often provide better observability and dependency management.

DEFINER and SQL SECURITY

Views and stored objects may execute with definer or invoker privileges. A dropped definer account can break objects; an overprivileged definer can expose data. Manage object ownership intentionally.

20. JSON in MySQL 8

Creating and Inserting JSON

SQL

```
CREATE TABLE product_attributes (
    product_id BIGINT PRIMARY KEY,
    attributes JSON NOT NULL,
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

INSERT INTO product_attributes VALUES
(1001, JSON_OBJECT('color', 'blue', 'sizes', JSON_ARRAY('S', 'M', 'L'),
    'material', 'cotton'));
```

MySQL validates documents stored in a JSON column.

Extracting Values

SQL

```
SELECT
    attributes -> '$.color' AS json_color,
    attributes ->> '$.color' AS text_color,
    JSON_EXTRACT(attributes, '$.sizes[0]') AS first_size,
    JSON_UNQUOTE(JSON_EXTRACT(attributes, '$.material')) AS material
FROM product_attributes;
```

-> returns JSON; ->> returns unquoted text. A missing path, JSON null, and SQL NULL are distinct concepts that should be tested explicitly.

Modifying JSON

SQL

```
UPDATE product_attributes
SET attributes = JSON_SET(attributes, '$.color', 'navy', '$.season', 'summer')
WHERE product_id = 1001;

UPDATE product_attributes
SET attributes = JSON_REMOVE(attributes, '$.season')
WHERE product_id = 1001;
```

- JSON_SET: insert or replace path.
- JSON_INSERT: insert only when absent.
- JSON_REPLACE: replace only when present.
- JSON_REMOVE: remove path.

Searching JSON

SQL

```
SELECT product_id
FROM product_attributes
WHERE JSON_CONTAINS(attributes, JSON_QUOTE('M'), '$.sizes');

SELECT product_id
FROM product_attributes
WHERE JSON_CONTAINS_PATH(attributes, 'one', '$.material', '$.brand');
```

JSON_TABLE

JSON_TABLE converts document elements into relational rows:

```
SELECT pa.product_id, jt.size_code
FROM product_attributes AS pa
JOIN JSON_TABLE(
    pa.attributes,
    '$.sizes[*]' COLUMNS (
        size_code VARCHAR(10) PATH '$'
    )
) AS jt;
```

SQL

JSON Aggregation

```
SELECT
    category_id,
    JSON_ARRAYAGG(product_name) AS product_names
FROM products
GROUP BY category_id;

SELECT JSON_OBJECTAGG(product_id, product_name)
FROM products;
```

SQL

Indexing JSON

Frequently filtered attributes should often become ordinary columns. A generated column can expose and index a JSON path:

```
ALTER TABLE product_attributes
ADD COLUMN color VARCHAR(30)
    GENERATED ALWAYS AS (attributes ->> '$.color') STORED,
ADD INDEX idx_product_color (color);
```

SQL

Use JSON for flexible, sparse attributes, not to avoid modeling stable relationships or constraints.

21. Indexes and the InnoDB Storage Model

An index is an ordered access structure that can reduce rows examined. It is not free: indexes consume storage and memory and add work to INSERT, UPDATE, and DELETE.

InnoDB Primary and Secondary Indexes

InnoDB organizes table rows by the primary key (the clustered index). Secondary index leaf entries contain the indexed columns plus the primary-key value, which is used to find the full row.

Consequences:

- Keep primary keys stable and reasonably compact.
- Secondary indexes become larger when the primary key is large.
- A lookup through a non-covering secondary index may require a second primary-key lookup.

Creating Indexes

```
CREATE INDEX idx_orders_customer_date
  ON orders(customer_id, order_at);

CREATE UNIQUE INDEX uq_sellers_code
  ON sellers(seller_code);

DROP INDEX idx_orders_customer_date ON orders;
```

SQL

Composite Index and Leftmost Prefix

For index (customer_id, order_at, status), useful leading prefixes are:

```
(customer_id)
(customer_id, order_at)
(customer_id, order_at, status)
```

TEXT

It generally cannot efficiently seek by order_at alone because the leading customer_id is absent. The optimizer can still scan an index in some cases, but the leftmost-prefix rule guides design.

Equality, Range, and Ordering

Consider:

```
SELECT order_id, order_at, status
FROM orders
WHERE customer_id = 501
  AND order_at >= '2026-01-01'
ORDER BY order_at DESC
LIMIT 20;
```

SQL

Index (customer_id, order_at) supports equality on customer, range on date, and ordering by date. Add columns only when workload evidence justifies the write/storage cost.

Once a range condition is used on one index part, later parts often cannot be used for further index seeking, though they may still help filtering or covering.

Covering Index

An index covers a query when all required columns can be obtained from the index without reading the base row.

```
CREATE INDEX idx_orders_customer_date_status
ON orders(customer_id, order_at, status);
```

SQL

Covering can improve read performance but wide indexes are expensive. Do not duplicate many columns only to chase covering status.

Selectivity and Cardinality

An index on a low-cardinality flag such as `active_flag` may be unhelpful alone when most rows share the same value. Combined with a selective date or category it may help. Measure plans and execution, not rules of thumb alone.

Specialized Indexes

- Unique index: enforces uniqueness and supports lookup.
- Prefix index: indexes an initial portion of long strings.
- Functional index: indexes an expression in supported MySQL 8 versions.
- FULLTEXT: token-based text search.
- SPATIAL: geographic/geometric operations.
- Multi-valued indexes: specialized indexing for JSON arrays in supported versions.

```
CREATE INDEX idx_customer_email_lower
ON customers ((LOWER(email)));
```

SQL

Verify exact version support and expression requirements.

Sargability

A predicate is sargable when an index can be used effectively to search it.

Less sargable:

```
WHERE DATE(order_at) = '2026-07-18'
WHERE YEAR(order_at) = 2026
WHERE CAST(customer_id AS CHAR) = '501'
WHERE product_name LIKE '%phone'
```

SQL

Better:

```
WHERE order_at >= '2026-07-18'
  AND order_at < '2026-07-19'

WHERE order_at >= '2026-01-01'
  AND order_at < '2027-01-01'

WHERE customer_id = 501
WHERE product_name LIKE 'phone%'
```

SQL

Avoid implicit casts between joined keys; align their data types.

22. EXPLAIN and Query Optimization

EXPLAIN

SQL

```
EXPLAIN
SELECT o.order_id, o.order_at
FROM orders AS o
WHERE o.customer_id = 501
      AND o.order_at >= '2026-01-01'
ORDER BY o.order_at DESC
LIMIT 20;
```

Important fields include:

Field	Meaning
table	table or derived step being accessed
type	access method, from broad scan to key lookup
possible_keys	indexes considered
key	chosen index
key_len	amount of index used
ref	values used for lookup
rows	estimated rows examined
filtered	estimated percentage surviving condition
Extra	additional operations and clues

Common access types include ALL (full scan), index (full index scan), range, ref, eq_ref, const, and system. A full scan is not automatically bad for a small table or query returning most rows.

Using temporary and Using filesort indicate work for grouping/sorting but are not automatic failures. Determine actual cost.

EXPLAIN ANALYZE

SQL

```
EXPLAIN ANALYZE
SELECT ...;
```

This executes the query and reports actual timing and row counts. Use it carefully on expensive or write-related statements. Compare estimated versus actual rows; large errors can suggest stale statistics, data skew, or correlated predicates.

FORMAT JSON

SQL

```
EXPLAIN FORMAT=JSON SELECT ...;
```

JSON exposes detailed cost and plan structure useful for advanced diagnosis.

Optimization Workflow

1. Confirm the query is logically correct and output grain is right.

2. Establish baseline duration, rows examined, and frequency.
3. Inspect EXPLAIN ANALYZE where safe.
4. Identify the expensive operator or row explosion.
5. Reduce unnecessary input early without changing semantics.
6. Make predicates sargable and data types compatible.
7. Add or modify an index only when the access pattern supports it.
8. Retest with representative data and concurrency.
9. Compare write/storage impact and regression risk.
10. Document the evidence.

Common Improvements

SQL

```
-- Select only required columns, not *.
-- Prefer UNION ALL when duplicate removal is unnecessary.
-- Pre-aggregate each fact source before joining.
-- Replace accidental DISTINCT with a correct relationship.
-- Use EXISTS for existence checks.
-- Avoid repeated correlated work when a grouped join is clearer.
-- Filter on raw indexed date ranges rather than functions on the date column.
```

Statistics and Histograms

SQL

```
ANALYZE TABLE orders;
ANALYZE TABLE orders UPDATE HISTOGRAM ON status WITH 10 BUCKETS;
ANALYZE TABLE orders DROP HISTOGRAM ON status;
```

Statistics help the optimizer estimate cardinality. Histograms can help skewed non-indexed columns. They do not replace appropriate indexes.

Partitioning Awareness

Partitioning divides a table physically according to a rule, potentially enabling partition pruning. It adds design and operational constraints and is not a universal performance solution. Indexes, correct queries, and data lifecycle design remain necessary.

Operational Tools

- Slow query log: identifies statements exceeding configured thresholds.
- Performance Schema: instruments waits, statements, stages, locks, and resources.
- SHOW PROCESSLIST: current sessions and work.
- SHOW ENGINE INNODB STATUS: InnoDB diagnostics, including recent deadlock data.

MySQL 8 removed the old query cache. Do not follow tuning advice that depends on it.

23. Security, Privacy, and Safe SQL

Preventing SQL Injection

Never concatenate untrusted input into SQL:

```
BAD: "SELECT * FROM users WHERE email = '" + user_input + "'"
```

TEXT

Use parameterized statements through the application driver:

```
SELECT customer_id, customer_name
FROM customers
WHERE email = ?;
```

SQL

Parameters represent values, not table names or keywords. For dynamic identifiers, use a strict server-side allow-list.

Least Privilege

Grant only required actions on required schemas/tables. Separate read-only analyst roles from data-changing and administrative roles. Review grants periodically.

```
CREATE ROLE ba_readonly;
GRANT SELECT, SHOW VIEW ON commerce_lab.* TO ba_readonly;
```

SQL

Avoid granting FILE, global administration, user management, or write privileges without a demonstrated need.

Sensitive Data

- Select only needed columns; do not export unnecessary personal data.
- Mask or tokenize identifiers in development and presentations.
- Avoid putting secrets or contact information in examples or logs.
- Apply retention and deletion policies.
- Encrypt connections and manage credentials through approved secret storage.
- Restrict bulk export and LOCAL INFILE capabilities.
- Audit access to sensitive datasets.

Backups and Recovery Awareness

A backup is useful only if restoration is tested. Production systems commonly combine full backups with binary logs for point-in-time recovery. Analysts should understand recovery constraints before destructive operations, even if infrastructure teams own backups.

Prepared Statement Example

Inside MySQL, dynamic SQL can use PREPARE, but application parameters are usually preferable:

SQL

```
PREPARE stmt FROM  
  'SELECT order_id, status FROM orders WHERE customer_id = ?';  
SET @customer_id = 501;  
EXECUTE stmt USING @customer_id;  
DEALLOCATE PREPARE stmt;
```

24. Data Quality, Debugging, and Reconciliation

Correct SQL can faithfully compute incorrect source data. A professional analyst validates both logic and data.

Incremental Query Development

Build complex queries in stages:

1. Inspect a small sample of each source.
2. Confirm keys, data types, NULLs, and table grain.
3. Apply base filters and record row counts.
4. Add one join at a time and compare counts.
5. Aggregate only after join behavior is understood.
6. Inspect intermediate CTEs independently.
7. Reconcile final metrics to trusted controls.
8. Test edge cases with tiny hand-calculated data.

Duplicate Detection

```
SELECT email, COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

SQL

For a supposed composite key:

```
SELECT order_id, product_id, COUNT(*) AS occurrences
FROM raw_order_items
GROUP BY order_id, product_id
HAVING COUNT(*) > 1;
```

SQL

Do not delete duplicates until you define which row is authoritative and whether the records are truly duplicates.

Missing and Invalid Values

```
SELECT
  COUNT(*) AS rows_total,
  SUM(city IS NULL) AS null_city,
  SUM(TRIM(COALESCE(city, '')) = '') AS blank_or_null_city,
  SUM(signup_at > NOW()) AS future_signup,
  SUM(status NOT IN ('active', 'inactive', 'blocked')) AS invalid_status
FROM customers;
```

SQL

Referential Gaps

Even when foreign keys are not enforced in a warehouse, check for orphan facts:

SQL

```
SELECT COUNT(*) AS orphan_orders
FROM orders AS o
LEFT JOIN customers AS c ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

Grain and Fanout Checks

SQL

```
SELECT
  COUNT(*) AS joined_rows,
  COUNT(DISTINCT o.order_id) AS orders,
  COUNT(DISTINCT CONCAT(oi.order_id, ':', oi.product_id)) AS order_items
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id;
```

After each join, know which count should increase and why.

Reconciliation

Compare independent calculations:

SQL

```
WITH item_total AS (
  SELECT order_id,
         SUM(quantity * unit_price - discount_amount) AS value_from_items
  FROM order_items
  GROUP BY order_id
), summary_compare AS (
  SELECT s.order_id,
         s.recorded_order_value,
         i.value_from_items,
         s.recorded_order_value - i.value_from_items AS difference
  FROM order_summary AS s
  JOIN item_total AS i ON i.order_id = s.order_id
)
SELECT
  COUNT(*) AS compared_orders,
  SUM(ABS(difference) > 0.01) AS mismatched_orders,
  SUM(difference) AS net_difference,
  MAX(ABS(difference)) AS largest_absolute_difference
FROM summary_compare;
```

A net difference of zero can hide offsetting positive and negative errors, so inspect absolute differences too.

Date and Time Debugging

Check:

- The business time zone.
- Whether boundaries are inclusive or exclusive.
- Whether date means event, ingestion, payment, delivery, or reporting date.
- Daylight-saving transitions where relevant.
- Late events and backfills.
- Whether the latest day is incomplete.

SQL

```
SELECT MIN(event_at), MAX(event_at), COUNT(*)
FROM events
```

SQL

```
WHERE event_at >= '2026-07-18'  
AND event_at < '2026-07-19';
```

SQL Errors and Warnings

SQL

```
SHOW WARNINGS;  
SELECT @@sql_mode;
```

Common error classes:

- Syntax error: malformed statement or reserved identifier.
- Unknown/ambiguous column: wrong scope or unqualified duplicate name.
- Constraint error: duplicate key, missing parent, invalid check.
- Cardinality error: scalar subquery returned multiple rows.
- Conversion/truncation warning: value does not fit expected type.
- Permission error: account lacks privilege.
- Lock timeout/deadlock: concurrency conflict.

Reduce a failing query to the smallest reproducing example. Solve correctness before performance.

Data Quality Scorecard

SQL

```
SELECT 'customers' AS dataset,  
       COUNT(*) AS rows_total,  
       COUNT(DISTINCT customer_id) AS distinct_keys,  
       SUM(customer_name IS NULL) AS missing_required,  
       SUM(signup_at > NOW()) AS invalid_dates  
FROM customers  
UNION ALL  
SELECT 'orders',  
       COUNT(*),  
       COUNT(DISTINCT order_id),  
       SUM(customer_id IS NULL),  
       SUM(order_at > NOW())  
FROM orders;
```


25. MySQL Dialect Differences and Portability

SQL concepts travel across systems; syntax and behavior do not always.

Common Differences

Concept	MySQL	PostgreSQL	SQL Server	Oracle / warehouse notes
Limit rows	LIMIT 10 OFFSET 20	same	OFFSET...FETCH, TOP	FETCH FIRST; warehouses vary
Auto ID	AUTO_INCREMENT	identity/sequence	IDENTITY	identity/sequence
Null replacement	IFNULL, COALESCE	COALESCE	ISNULL, COALESCE	NVL, COALESCE
String aggregate	GROUP_CONCAT	STRING_AGG	STRING_AGG	LISTAGG
Date add	DATE_ADD(d, INTERVAL 1 DAY)	d + INTERVAL '1 day'	DATEADD(day,1,d)	varies
Identifier quote	backtick	double quote	brackets/double quote	double quote
Upsert	ON DUPLICATE KEY	ON CONFLICT	MERGE patterns	MERGE; warehouse- specific
Boolean	TINYINT(1) synonym	native boolean	bit	varies
Null-safe equality	<=>	IS NOT DISTINCT FROM	expanded logic	varies

Important MySQL-Specific Behavior

- CONCAT returns NULL when any argument is NULL; CONCAT_WS skips NULL arguments.
- IF and IFNULL are convenient but less portable than CASE and COALESCE.
- LIMIT is MySQL/PostgreSQL-style, not standard everywhere.
- UPDATE ... JOIN and DELETE ... JOIN differ across systems.
- ONLY_FULL_GROUP_BY prevents ambiguous grouping.
- CTEs and window functions require MySQL 8+, not MySQL 5.7.
- MySQL lacks native QUALIFY, general materialized views, and native FULL OUTER JOIN.
- Identifier and string case sensitivity depends on platform settings, collation, and object type.
- Double quotes usually represent strings unless ANSI_QUOTES SQL mode is enabled; use single quotes for strings.

Portable Style

Prefer:

```
CASE WHEN ... THEN ... ELSE ... END
COALESCE(value, fallback)
explicit JOIN ... ON
single quotes for strings
explicit column lists
half-open timestamp ranges
```

SQL

Avoid relying on implicit conversions, ungrouped selected columns, undocumented ordering, or vendor-specific shortcuts when portability matters.

26. Advanced Analytical Patterns

Median in MySQL 8

MySQL lacks a simple standard MEDIAN() aggregate. Rank values and average the middle position(s):

```
WITH ranked AS (
  SELECT
    list_price,
    ROW_NUMBER() OVER (ORDER BY list_price) AS rn,
    COUNT(*) OVER () AS n
  FROM products
  WHERE list_price IS NOT NULL
)
SELECT AVG(list_price) AS median_price
FROM ranked
WHERE rn IN (FLOOR((n + 1) / 2), FLOOR((n + 2) / 2));
```

SQL

For odd n, both expressions identify the same middle row; for even n, they identify the two middle rows.

Percentile by Nearest Rank

```
WITH ranked AS (
  SELECT delivery_hours,
    ROW_NUMBER() OVER (ORDER BY delivery_hours) AS rn,
    COUNT(*) OVER () AS n
  FROM deliveries
  WHERE delivery_hours IS NOT NULL
)
SELECT MIN(delivery_hours) AS p95_delivery_hours
FROM ranked
WHERE rn >= CEIL(0.95 * n);
```

SQL

Different percentile definitions interpolate differently. Match the organization's standard before comparing tools.

As-Of Join / Effective-Dated Dimension

Find the product category version valid when an order occurred:

```
SELECT
  oi.order_id,
  oi.product_id,
  d.category_name
FROM order_items AS oi
JOIN orders AS o ON o.order_id = oi.order_id
JOIN product_dimension_history AS d
  ON d.product_id = oi.product_id
  AND o.order_at >= d.effective_from
  AND o.order_at < COALESCE(d.effective_to, '9999-12-31');
```

SQL

Ensure effective ranges do not overlap for the same entity.

Overlapping Date Ranges

Two half-open ranges [start_a, end_a) and [start_b, end_b) overlap when:

```
start_a < end_b AND start_b < end_a
```

TEXT

```
SELECT a.promotion_id, b.promotion_id
FROM promotions AS a
JOIN promotions AS b
  ON a.promotion_id < b.promotion_id
 AND a.start_at < b.end_at
 AND b.start_at < a.end_at;
```

SQL

First and Last Event per Entity

```
WITH ranked AS (
  SELECT e.*,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id ORDER BY event_at, event_id
         ) AS first_rn,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id ORDER BY event_at DESC, event_id DESC
         ) AS last_rn
  FROM events AS e
)
SELECT customer_id,
       MAX(CASE WHEN first_rn = 1 THEN event_at END) AS first_event_at,
       MAX(CASE WHEN last_rn = 1 THEN event_at END) AS last_event_at
FROM ranked
GROUP BY customer_id;
```

SQL

Time Between Events

```
SELECT
  customer_id,
  order_id,
  order_at,
  LAG(order_at) OVER (
    PARTITION BY customer_id ORDER BY order_at, order_id
  ) AS previous_order_at,
  TIMESTAMPDIFF(DAY,
    LAG(order_at) OVER (
      PARTITION BY customer_id ORDER BY order_at, order_id
    ), order_at) AS days_since_previous
FROM orders;
```

SQL

Relational Division: Customers Who Bought Every Required Category

```
SELECT o.customer_id
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN products AS p ON p.product_id = oi.product_id
WHERE p.category_id IN (2, 5, 8)
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.category_id) = 3;
```

SQL

If the required set is stored in a table, compare with its count rather than hard-coding
3.

Running Balance with Signed Transactions

SQL

```
SELECT
  account_id,
  transaction_at,
  transaction_id,
  amount,
  SUM(amount) OVER (
    PARTITION BY account_id
    ORDER BY transaction_at, transaction_id
    ROWS UNBOUNDED PRECEDING
  ) AS running_balance
FROM account_transactions;
```

Market Basket Pairs

SQL

```
SELECT
  a.product_id AS product_a,
  b.product_id AS product_b,
  COUNT(DISTINCT a.order_id) AS orders_together
FROM order_items AS a
JOIN order_items AS b
  ON b.order_id = a.order_id
  AND b.product_id > a.product_id
GROUP BY a.product_id, b.product_id
HAVING COUNT(DISTINCT a.order_id) >= 10
ORDER BY orders_together DESC;
```

The strict product ordering avoids mirrored pairs and self-pairs.

A/B Experiment Summary

Suppose each user has one assigned variant and outcome events occur after assignment.

SQL

```
WITH outcomes AS (
  SELECT
    a.variant,
    a.customer_id,
    MAX(e.event_name = 'purchase') AS converted,
    SUM(CASE WHEN e.event_name = 'purchase'
      THEN CAST(e.properties ->> '$.order_value' AS DECIMAL(12,2))
      ELSE 0 END) AS revenue
  FROM experiment_assignments AS a
  LEFT JOIN events AS e
    ON e.customer_id = a.customer_id
    AND e.event_at >= a.assigned_at
    AND e.event_at < a.assigned_at + INTERVAL 14 DAY
  WHERE a.experiment_name = 'checkout_v2'
  GROUP BY a.variant, a.customer_id
)
SELECT
  variant,
  COUNT(*) AS assigned_users,
  SUM(converted) AS converters,
  100.0 * SUM(converted) / NULLIF(COUNT(*), 0) AS conversion_pct,
  SUM(revenue) / NULLIF(COUNT(*), 0) AS revenue_per_assigned_user
FROM outcomes
```

```
GROUP BY variant;
```

SQL produces experiment summaries; statistical significance and uncertainty still require valid randomization, sample-size reasoning, and an appropriate statistical test. Check sample-ratio mismatch, assignment integrity, pre-experiment balance, guardrail metrics, novelty effects, and multiple testing.

Snapshot versus Event Data

An event table records changes or actions. A snapshot table records state at a point in time. Summing daily inventory snapshots across dates is normally invalid; inventory is semi-additive. Use the latest snapshot for end-of-period stock or average snapshots for average stock, depending on the question.

27. SQL Interview Patterns with Solutions

In an interview, begin by clarifying table grain, desired output, ties, NULL handling, and date boundaries. Then state an approach before typing.

Pattern 1: Nth Highest Value

Second-highest distinct salary:

```
WITH ranked AS (
  SELECT employee_id, salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
  FROM employees
)
SELECT employee_id, salary
FROM ranked
WHERE salary_rank = 2;
```

SQL

DENSE_RANK returns every employee tied at the second distinct salary. ROW_NUMBER = 2 returns exactly one row. Clarify which is required.

Alternative scalar result:

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

SQL

Pattern 2: Above Group Average

```
WITH scored AS (
  SELECT e.*,
         AVG(salary) OVER (PARTITION BY department_id) AS department_avg
  FROM employees AS e
)
SELECT employee_id, department_id, salary, department_avg
FROM scored
WHERE salary > department_avg;
```

SQL

A correlated subquery also works, but the window approach calculates and exposes the comparison value clearly.

Pattern 3: Top Three per Category

```
WITH ranked AS (
  SELECT p.*,
         DENSE_RANK() OVER (
           PARTITION BY category_id
           ORDER BY list_price DESC
         ) AS price_rank
  FROM products AS p
)
SELECT * FROM ranked WHERE price_rank <= 3;
```

SQL

Use ROW_NUMBER for exactly three products per category and add a deterministic tie-breaker.

Pattern 4: Customers with No Orders

SQL

```
SELECT c.customer_id, c.customer_name
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1 FROM orders AS o
    WHERE o.customer_id = c.customer_id
);
```

Mention why NOT EXISTS is safer than NOT IN when NULL may appear.

Pattern 5: Latest Record per Entity

SQL

```
WITH ranked AS (
    SELECT s.*,
        ROW_NUMBER() OVER (
            PARTITION BY order_id
            ORDER BY status_at DESC, status_history_id DESC
        ) AS rn
    FROM order_status_history AS s
)
SELECT * FROM ranked WHERE rn = 1;
```

The tie-breaker prevents nondeterministic results.

Pattern 6: Find and Safely Remove Duplicates

Inspect first:

SQL

```
SELECT normalized_email, COUNT(*)
FROM customer_staging
GROUP BY normalized_email
HAVING COUNT(*) > 1;
```

Choose a survivor rule:

SQL

```
WITH marked AS (
    SELECT staging_id,
        ROW_NUMBER() OVER (
            PARTITION BY normalized_email
            ORDER BY updated_at DESC, staging_id DESC
        ) AS rn
    FROM customer_staging
)
SELECT * FROM marked WHERE rn > 1;
```

Before deletion, persist the duplicate keys for audit, verify foreign-key effects, and test inside a transaction. MySQL's restriction on modifying a table read from in the same query can require an extra derived layer or temporary table.

Pattern 7: First and Repeat Purchase

SQL

```
WITH sequenced AS (
    SELECT o.*,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id ORDER BY order_at, order_id
        ) AS purchase_number
```


SQL

```

FROM orders AS o
WHERE status = 'delivered'
)
SELECT
    DATE(order_at) AS order_date,
    SUM(purchase_number = 1) AS first_purchases,
    SUM(purchase_number > 1) AS repeat_purchases
FROM sequenced
GROUP BY DATE(order_at);

```

Clarify whether a previously cancelled order counts as first purchase; usually the chosen eligible status determines it.

Pattern 8: Consecutive Days

Use the gaps-and-islands technique from Chapter 15. A common follow-up asks for customers with at least seven consecutive active days:

SQL

```

WITH dates AS (
    SELECT DISTINCT customer_id, DATE(event_at) AS dt FROM events
), numbered AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY dt) AS rn
    FROM dates
), islands AS (
    SELECT *, DATE_SUB(dt, INTERVAL rn DAY) AS grp FROM numbered
)
SELECT customer_id, MIN(dt) AS start_dt, MAX(dt) AS end_dt, COUNT(*) AS days
FROM islands
GROUP BY customer_id, grp
HAVING COUNT(*) >= 7;

```

Pattern 9: Month-over-Month Growth

Aggregate to month first, then use LAG. Explain how you handle missing months and zero previous value.

SQL

```

WITH m AS (
    SELECT DATE_FORMAT(order_at, '%Y-%m-01') AS month_start,
           COUNT(*) AS orders
    FROM orders
    GROUP BY DATE_FORMAT(order_at, '%Y-%m-01')
), x AS (
    SELECT *, LAG(orders) OVER (ORDER BY month_start) AS prev_orders
    FROM m
)
SELECT *, 100.0 * (orders - prev_orders) / NULLIF(prev_orders, 0) AS mom_pct
FROM x;

```

Pattern 10: Percent of Total

SQL

```

SELECT category_id, revenue,
       100.0 * revenue / NULLIF(SUM(revenue) OVER (), 0) AS revenue_share_pct
FROM category_revenue;

```

Pattern 11: Running Total

SQL

```
SELECT metric_date, daily_revenue,
       SUM(daily_revenue) OVER (
         ORDER BY metric_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_revenue
FROM daily_metrics;
```

Explain ROWS versus the default frame and include missing dates when the business expects calendar continuity.

Pattern 12: Retention

State cohort event, return event, and time bucket before writing. Then build: eligible activity -> cohort -> month offset -> distinct active customers -> divide by month-zero cohort size.

Pattern 13: Funnel Conversion

Clarify entity (user/session/order), strict ordering, window, duplicate events, and denominator. Use per-entity flags before aggregation to prevent repeat events from inflating counts.

Pattern 14: Join Inflation

Question: orders have items and payments; calculate value and paid amount. Answer: aggregate items and payments separately to one row per order, then join. Describe the many-to-many fanout if joined raw.

Pattern 15: Find Missing Sequence Values

For gaps between integer IDs:

SQL

```
WITH RECURSIVE seq AS (
  SELECT MIN(id) AS n FROM records
  UNION ALL
  SELECT n + 1 FROM seq
  WHERE n < (SELECT MAX(id) FROM records)
)
SELECT n AS missing_id
FROM seq
LEFT JOIN records AS r ON r.id = seq.n
WHERE r.id IS NULL;
```

IDs may legitimately have gaps; do not treat auto-increment gaps as corruption automatically.

Pattern 16: Overlapping Intervals

Use $a.start < b.end$ AND $b.start < a.end$ for half-open ranges. Add $a.id < b.id$ in a self-join to remove mirrored duplicates.

Pattern 17: Conditional KPI Report

SQL

```
SELECT
  DATE(order_at) AS dt,
  COUNT(*) AS orders,
  SUM(status = 'delivered') AS delivered,
  SUM(status = 'cancelled') AS cancelled,
```

SQL

```
100.0 * SUM(status = 'cancelled') / NULLIF(COUNT(*), 0) AS cancel_pct
FROM orders
GROUP BY DATE(order_at);
```

For portable SQL, replace boolean sums with CASE.

Pattern 18: Recursive Hierarchy

Explain anchor, recursive member, relationship, termination, depth, and cycle risk. Return path and depth so the result is auditable.

Pattern 19: Median

Use row number and total count, average one or two middle rows. Ask how duplicates and NULLs are handled.

Pattern 20: Data Validation

When asked "How do you know your result is correct?", answer with concrete checks:

- Reconcile total rows and totals before/after transformations.
- Check unique key assumptions.
- Inspect unmatched joins.
- Compare COUNT() with COUNT(DISTINCT key).
- Hand-calculate a tiny sample.
- Test NULLs, no matches, duplicates, ties, zero denominator, and boundary timestamps.
- Compare with an independent source or prior period.

Interview Communication Template

TEXT

1. Clarify: "Should returned orders be excluded, and which date defines the month?"
2. State grain: "I will first create one row per delivered order."
3. State approach: "Aggregate items, join customer attributes, then use LAG."
4. Write readable SQL with aliases and stages.
5. Test edge cases and explain complexity/index needs.
6. Summarize the business meaning, not only the query.

28. Business Analyst Cases for E-Commerce

These cases mirror the open-ended, metric-ownership work described in the attached Business Analyst role. They are practice scenarios, not claims about any company's actual interview questions.

Case 1: Orders Dropped 12% Yesterday

Clarify:

- Compared with previous day, same weekday, or forecast?
- Orders placed, paid, or delivered?
- Entire platform or a region/channel/device?
- Is yesterday complete in the business time zone?

Investigation structure:

1. Validate event pipeline, dashboard freshness, and definition changes.
2. Decompose orders = traffic x conversion.
3. Segment by platform, app version, acquisition channel, city, category, seller, payment method, and new/existing user.
4. Inspect funnel stage rates and technical failure metrics.
5. Check internal releases, pricing, inventory, campaigns, and external seasonality.
6. Quantify contribution of each segment to total decline.

Contribution query outline:

```
WITH daily AS (
  SELECT DATE(order_at) AS dt, payment_method, COUNT(*) AS orders
  FROM orders
  WHERE order_at >= '2026-07-16' AND order_at < '2026-07-18'
  GROUP BY DATE(order_at), payment_method
)
SELECT payment_method,
  SUM(CASE WHEN dt = '2026-07-16' THEN orders ELSE 0 END) AS previous_orders,
  SUM(CASE WHEN dt = '2026-07-17' THEN orders ELSE 0 END) AS current_orders,
  SUM(CASE WHEN dt = '2026-07-17' THEN orders ELSE 0 END)
  - SUM(CASE WHEN dt = '2026-07-16' THEN orders ELSE 0 END) AS contribution_change
FROM daily
GROUP BY payment_method
ORDER BY contribution_change;
```

SQL

Case 2: Conversion Increased but Returns Also Increased

Do not declare success from conversion alone. Build an outcome tree:

```
Traffic -> order conversion -> delivered rate -> retained/net value -> contribution margin
Guardrails: returns, cancellations, complaints, delivery time, seller quality
```

TEXT

Compare variants or periods at consistent maturity. Recent orders may not have had enough time to return, causing right-censoring. Use a fixed observation window per order.

Case 3: Discount Campaign Evaluation

Metrics:

- Incremental orders and customers, not raw campaign orders.
- Incremental net value and contribution after discount.
- Cannibalization from full-price sales.
- New-customer quality and later retention.
- Seller/category concentration.
- Return and cancellation guardrails.

A/B testing is preferred where feasible. A before-after comparison alone confounds seasonality and other changes.

Case 4: Seller Activation Declined

Define activation, for example: seller completes onboarding and lists one valid product within seven days. Build a seller funnel:

```
registration -> verification -> catalog upload -> approved listing -> first live product -> first order
```

TEXT

Use one row per seller with stage timestamps; measure stage completion and time-to-stage. Segment by acquisition source, geography, category, device, and onboarding version.

Case 5: Delivery Time Increased in One Region

Separate processing, pickup, line-haul, last-mile, and customer-delay components. Compare percentiles, not only averages; a small long-tail can drive dissatisfaction.

```
SELECT region,
       AVG(TIMESTAMPDIFF(HOUR, placed_at, delivered_at)) AS avg_hours
FROM deliveries
WHERE delivered_at IS NOT NULL
GROUP BY region;
```

SQL

For median/p95, use the percentile patterns in Chapter 26. Check mix shifts in seller, category, distance, courier, weather, and promised service level.

Case 6: Build a Daily Pod Dashboard

A useful dashboard includes:

- North-star outcome and driver tree.
- Actual, target, previous comparable period, and change.
- New versus existing customer split.
- Funnel stage rates.
- Quality/guardrail metrics.
- Top contributing segments to movement.
- Data freshness and metric definitions.

Avoid presenting dozens of metrics without a decision hierarchy.

Insight-to-Action Template

TEXT

Observation: North-region purchase conversion fell 8% week over week.
Localization: 72% of the decline came from new Android users on app version X.
Mechanism: payment-success rate dropped after checkout release Y.
Recommendation: pause rollout for affected users and investigate gateway routing.
Expected result: recover payment success and 5-7% conversion loss.
Guardrails: duplicate payments, latency, refunds, and support contacts.
Validation: compare rollback cohort with unaffected versions over 48 hours.

A good analyst distinguishes observed evidence, plausible hypothesis, and causal conclusion.

29. Practice Curriculum and Exercises

Use the schema in the preface. Attempt each problem before reading hints or solutions. Reattempt failed questions after one day and one week.

Level 1: Fundamentals

1. Return product name and list price for active products.
2. Find orders placed in January 2026 using a half-open date range.
3. List distinct shipping cities alphabetically, with NULL last.
4. Return the five most expensive products with deterministic ordering.
5. Return products ranked 6-10 by price using LIMIT/OFFSET.
6. Find product names beginning with Pro.
7. Find codes where exactly one character follows WALL-.
8. Label products Budget, Mid-range, or Premium using CASE.
9. Replace missing city with Unknown for display.
10. Calculate gross and net item value.

Level 2: Aggregation and Joins

11. Count orders by status.
12. Calculate total employee years by building; explain why SUM, not COUNT.
13. Calculate monthly delivered order count.
14. Return each order with customer name.
15. Return every customer and their order count, including zero.
16. Find customers with no orders using NOT EXISTS.
17. Calculate sales by category without double counting.
18. Find categories with at least ten active products.
19. Calculate cancellation rate by payment method.
20. Return domestic and international sales with the movie title.

Level 3: CTEs, Subqueries, and Windows

21. Find products priced above category average.
22. Return the top three products by price per category.
23. Return each customer's latest delivered order.
24. Calculate cumulative daily revenue.
25. Calculate seven-day moving average orders with a calendar spine.
26. Calculate month-over-month revenue growth.
27. Find customers whose second delivered order occurred within 30 days of first.
28. Find the second-highest distinct seller revenue.
29. Find products contributing to the first 80% of sales.

30. Generate every date in the current month using recursive CTE.

Level 4: Analytics Cases

31. Build a session-based view-to-purchase funnel.
32. Build a monthly cohort-retention table.
33. Create RFM features and assign segments.
34. Sessionize event data with a 30-minute inactivity threshold.
35. Identify customer activity streaks of seven days or more.
36. Compare conversion for new versus existing customers.
37. Diagnose order decline by calculating segment contribution.
38. Measure delivered value net of returned orders under a documented definition.
39. Build an experiment result table with conversion and revenue per assigned user.
40. Create a seller activation funnel with median time between stages.

Level 5: Design and Performance

41. Design normalized tables for sellers, catalogs, products, and listing approvals.
42. Convert order data into a simple star schema and state fact grain.
43. Diagnose a query that joins order items and payment attempts and inflates sales.
44. Propose an index for latest 20 orders of one customer and justify column order.
45. Rewrite WHERE DATE(order_at)=... as a sargable range.
46. Compare EXPLAIN ANALYZE before and after an index.
47. Find redundant indexes using metadata and workload evidence.
48. Create a read-only role for an analyst.
49. Write a safe transactional inventory decrement.
50. Write data-quality checks for keys, NULLs, dates, and referential gaps.

Capstone Project

Use an e-commerce dataset with orders, customers, items, products, sessions, and returns.

Deliverables:

1. A data dictionary stating every table's grain and keys.
2. At least 20 SQL queries covering joins, CTEs, windows, cohorts, funnel, and RCA.
3. A validation file with row-count, duplicate, NULL, and reconciliation checks.
4. An Excel or BI dashboard with five to eight decision-oriented metrics.
5. A one-page executive memo: three findings, evidence, action, impact, and guardrails.
6. A five-minute verbal walkthrough and a ten-minute technical explanation.

Self-Scoring Rubric

Dimension	0-1	2	3
Correctness	wrong/incomplete	works on simple data	handles edge cases and validates
Grain	unstated/wrong	implicit	explicit throughout

Readability	copied/opaque	understandable	staged, named, and documented
Business meaning	only numbers	basic interpretation	decision, impact, and
			guardrails
Performance	no awareness	generic claim	evidence from plan and row
			counts
Communication	cannot explain	explains syntax	explains assumptions and trade-
			offs

30. Master Syntax Cheat Sheet

Core SELECT

SQL

```
SELECT [DISTINCT]
      expression AS alias,
      aggregate_function(column)
FROM table_a AS a
[INNER|LEFT|RIGHT|CROSS] JOIN table_b AS b
  ON a.key = b.key
WHERE row_condition
GROUP BY grouping_expression
HAVING aggregate_condition
ORDER BY sort_expression [ASC|DESC]
LIMIT row_count OFFSET rows_to_skip;
```

Logical order: FROM/JOIN -> WHERE -> GROUP BY -> aggregates -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT.

Filter Templates

SQL

```
column = value
column <> value
column BETWEEN low AND high           -- inclusive
column IN (value1, value2)
column LIKE 'prefix%'
column LIKE '_x%'                     -- one char, then x, then anything
column IS NULL
EXISTS (SELECT 1 FROM ... WHERE correlation)

-- Safe timestamp period
occurred_at >= :start_at
AND occurred_at < :end_at
```

JOIN Decision Table

Need	Use
Only matched rows	INNER JOIN
Every left row, optional right data	LEFT JOIN
Every combination	CROSS JOIN
Same table in two roles	self join
At least one related row, no right columns	EXISTS
No related row	NOT EXISTS
Matches plus unmatched on both sides	FULL JOIN; emulate in MySQL

Aggregation Templates

SQL

```
COUNT(*)
COUNT(column)           -- excludes NULL
COUNT(DISTINCT column)
```

SQL

```

SUM(amount)
AVG(amount)
MIN(amount)
MAX(amount)

SUM(CASE WHEN condition THEN amount ELSE 0 END)
SUM(CASE WHEN condition THEN 1 ELSE 0 END)
100.0 * numerator / NULLIF(denominator, 0)
SUM(value * weight) / NULLIF(SUM(weight), 0)

```

CTE Template

SQL

```

WITH step_one AS (
    SELECT ...
), step_two AS (
    SELECT ... FROM step_one
)
SELECT ... FROM step_two;

```

Recursive CTE Template

SQL

```

WITH RECURSIVE r AS (
    SELECT ... -- anchor
    UNION ALL
    SELECT ... FROM source JOIN r ON ... -- recursive member
    WHERE termination_condition
)
SELECT * FROM r;

```

Window Templates

SQL

```

ROW_NUMBER() OVER (PARTITION BY group_col ORDER BY metric DESC, id)
RANK() OVER (PARTITION BY group_col ORDER BY metric DESC)
DENSE_RANK() OVER (PARTITION BY group_col ORDER BY metric DESC)
LAG(value) OVER (PARTITION BY entity ORDER BY occurred_at, id)
LEAD(value) OVER (PARTITION BY entity ORDER BY occurred_at, id)

SUM(value) OVER (
    PARTITION BY optional_group
    ORDER BY dt
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)

AVG(value) OVER (
    ORDER BY dt ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
)

```

Top N per Group

SQL

```

WITH ranked AS (
    SELECT t.*,
           ROW_NUMBER() OVER (
               PARTITION BY group_col ORDER BY metric DESC, id
           ) AS rn
    FROM source AS t
)

```

SQL

```
SELECT * FROM ranked WHERE rn <= :n;
```

Latest Row per Entity

SQL

```
WITH ranked AS (
  SELECT t.*,
         ROW_NUMBER() OVER (
           PARTITION BY entity_id
           ORDER BY updated_at DESC, record_id DESC
         ) AS rn
  FROM source AS t
)
SELECT * FROM ranked WHERE rn = 1;
```

Period-over-Period

SQL

```
WITH period_metrics AS (...), compared AS (
  SELECT *, LAG(metric) OVER (ORDER BY period_start) AS previous_metric
  FROM period_metrics
)
SELECT *,
       metric - previous_metric AS absolute_change,
       100.0 * (metric - previous_metric) / NULLIF(previous_metric, 0)
       AS growth_pct
FROM compared;
```

Safe Transaction

SQL

```
START TRANSACTION;
SELECT ... FOR UPDATE;
UPDATE ... WHERE precise_key_condition;
SELECT ROW_COUNT();
-- Validate
COMMIT; -- or ROLLBACK
```

DDL Quick Reference

SQL

```
CREATE DATABASE database_name;
USE database_name;
CREATE TABLE table_name (...);
ALTER TABLE table_name ADD COLUMN ...;
ALTER TABLE table_name MODIFY COLUMN ...;
ALTER TABLE table_name DROP COLUMN ...;
ALTER TABLE table_name ADD CONSTRAINT ...;
RENAME TABLE old_name TO new_name;
TRUNCATE TABLE table_name;
DROP TABLE table_name;
CREATE INDEX index_name ON table_name(col1, col2);
DROP INDEX index_name ON table_name;
```

DML Quick Reference

SQL

```
INSERT INTO table_name (col1, col2) VALUES (v1, v2), (v3, v4);
INSERT INTO target (col1, col2) SELECT col1, col2 FROM source;
```

```
UPDATE table_name SET col1 = expression WHERE condition;
DELETE FROM table_name WHERE condition;
```

Common Function Index

Category	Functions
NULL	COALESCE, IFNULL, NULLIF, IS NULL, <=>
Conditional	CASE, IF
String	CONCAT, CONCAT_WS, TRIM, SUBSTRING, REPLACE, LOWER, UPPER, LENGTH, CHAR_LENGTH
Numeric	ROUND, TRUNCATE, CEIL, FLOOR, ABS, MOD, POWER, SQRT
Date	NOW, CURRENT_DATE, DATE_ADD, DATE_SUB, DATEDIFF, TIMESTAMPDIFF, EXTRACT, DATE_FORMAT, STR_TO_DATE
Conversion	CAST, CONVERT
Aggregate	COUNT, SUM, AVG, MIN, MAX, GROUP_CONCAT
Window ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE
Window navigation	LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE
Distribution	PERCENT_RANK, CUME_DIST
JSON	JSON_OBJECT, JSON_ARRAY, JSON_EXTRACT, JSON_SET, JSON_TABLE, JSON_ARRAYAGG

Destructive Statement Safety Checklist

1. Confirm environment and schema.
2. Back up or establish recovery plan.
3. Run the exact WHERE predicate as SELECT.
4. Check count and representative keys.
5. Use a transaction where supported and appropriate.
6. Lock only required rows and keep work short.
7. Run change and inspect ROW_COUNT().
8. Reconcile totals and samples.
9. Commit only after validation; otherwise roll back.
10. Record who changed what, why, and when.

Interview Clarification Checklist

- What does one input and output row represent?
- Which date and time zone apply?
- Which statuses are eligible?
- How should NULLs and missing matches behave?
- Are duplicates possible or meaningful?
- How should ties be handled?
- Is the denominator users, sessions, orders, or events?

- Should periods with zero activity appear?
- Is exact MySQL syntax required?
- What scale and index assumptions are reasonable?

31. Common Mistakes and Their Fixes

Mistake	Why it fails	Better approach
column = NULL	comparison becomes UNKNOWN	column IS NULL
IN (1,5) for 1 through 5	matches only 1 and 5	BETWEEN 1 AND 5 or explicit list
COUNT(years) for total years	counts non-NULL rows	SUM(years)
Exact = 'Toy Story' for every sequel	only exact text matches	LIKE 'Toy Story%'
LIKE '_' misunderstood	underscore is one character	% is zero or more characters
Latest four without LIMIT	returns every sorted row	ORDER BY date DESC LIMIT 4
OFFSET interpreted as starting row	it skips rows first	skip N, then return LIMIT rows
Join sales without movie title	numbers lack entity context	include title or ID
LEFT JOIN filter in WHERE	removes unmatched rows	put right filter in ON when preserving left
COUNT() after LEFT JOIN	counts NULL-extended row	count a non-NULL right key
Raw items joined to raw payments	many-to-many fanout	aggregate each to order grain first
NOT IN with nullable subquery	NULL can make all results UNKNOWN	NOT EXISTS
BETWEEN full dates for timestamps	may miss final fractional time	half-open [start, next_start)
MONTH(date) across years	combines same month from all years	group by year-month/start-of-month
DISTINCT added blindly	may hide join bug	diagnose relationship and grain
LIMIT without ORDER BY	arbitrary subset	deterministic ORDER BY with tie-breaker
Filter window alias in WHERE	window computed later	CTE/derived table, then filter
LAST_VALUE default frame	often returns current-row value	specify UNBOUNDED FOLLOWING
Function on indexed date	often prevents range seek	compare raw column to range
Average of averages	ignores group sizes	recompute from total numerator/denominator

32. Glossary

Aggregate: function reducing multiple rows to a summary, such as SUM or COUNT.

Alias: temporary name assigned to a table or expression in a query.

Anti-join: operation returning rows with no match, usually NOT EXISTS.

B-tree: ordered index structure supporting equality, ranges, and ordered retrieval.

Cardinality: count of distinct values or relationship multiplicity, depending on context.

CTE: Common Table Expression, a named result scoped to one statement.

Collation: rules for comparing and sorting text.

Constraint: database-enforced rule such as primary key, foreign key, unique, or check.

Correlated subquery: subquery that references a value from its outer query.

Covering index: index containing everything needed by a query.

DDL/DML/DQL/TCL/DCL: command families for definition, modification, querying, transactions, and access control.

Derived table: subquery used in FROM.

Deterministic ordering: ordering fully specified so ties cannot change row sequence unpredictably.

Dimension: descriptive analytical entity such as customer, product, or date.

Fact: measurable event at a declared grain.

Fanout: multiplication of rows when joins encounter multiple matches.

Foreign key: child value referencing a parent key.

Frame: subset of rows around the current row used by an ordered window function.

Grain: what exactly one row represents.

Idempotent: operation that can be repeated without creating additional unintended effect.

Index selectivity: how strongly an indexed value narrows rows.

Inner join: returns matching combinations only.

Isolation level: degree to which concurrent transaction effects are separated.

Keyset pagination: fetch next page using the last ordered key rather than deep OFFSET.

Materialization: storing an intermediate query result during execution.

MVCC: Multi-Version Concurrency Control, serving snapshots from row versions.

Normalization: organizing data to reduce redundancy and anomalies.

NULL: unknown, missing, or not-applicable value; not zero or empty string.

OLAP/OLTP: analytical versus operational transaction processing workloads.

Outer join: preserves unmatched rows from one or both inputs.

Partition (window): group of rows over which a window function operates.

Partition (table): physical/logical division of table storage by a partitioning rule.

Predicate: boolean condition in WHERE, ON, HAVING, or CASE.

Primary key: chosen unique, non-NULL row identifier.

Recursive CTE: CTE that repeatedly references its previous result.

Sargable: predicate form that can use an index efficiently for search.

Scalar subquery: subquery expected to return one value.

Semi-join: existence test returning only left rows, usually EXISTS.

Star schema: fact table connected to denormalized dimensions.

Surrogate key: artificial identifier without business meaning.

Transaction: unit of work committed or rolled back as a whole.

Window function: calculation across related rows while retaining detail rows.

33. Reference and Continued Practice

Primary Documentation

- MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL SELECT statement: <https://dev.mysql.com/doc/refman/8.0/en/select.html>
- MySQL functions and operators:
<https://dev.mysql.com/doc/refman/8.0/en/functions.html>
- MySQL window functions:
<https://dev.mysql.com/doc/refman/8.0/en/window-functions.html>
- MySQL common table expressions: <https://dev.mysql.com/doc/refman/8.0/en/with.html>
- MySQL optimization and EXPLAIN:
<https://dev.mysql.com/doc/refman/8.0/en/optimization.html>
- MySQL InnoDB transactions:
<https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-model.html>

Learning and Practice Path

Use the MySQL beginner, intermediate, and advanced tutorial playlists shown in the attached Alex The Analyst screenshots, then move immediately to hands-on problems. A focused order is:

1. Finish SQLBolt fundamentals.
2. Complete MySQL Basics, Intermediate MySQL, and Advanced MySQL tutorial series.
3. Solve LeetCode SQL 50.
4. Add business-oriented DataLemur and StrataScratch questions.
5. Build one e-commerce analytics project using the capstone checklist.
6. Practice explaining every solution aloud under time pressure.

Useful practice sites:

- SQLBolt: <https://sqlbolt.com/>
- LeetCode SQL 50: <https://leetcode.com/studyplan/top-sql-50/>
- DataLemur SQL questions: <https://datalemur.com/sql-interview-questions>
- HackerRank SQL: <https://www.hackerrank.com/domains/sql>
- StrataScratch: <https://www.stratascratch.com/>

Final Readiness Standard

You are ready for an advanced Business Analyst SQL round when you can:

- Translate an ambiguous metric into grain, population, date, numerator, and denominator.
- Solve medium join, aggregation, CTE, and window problems without copying.
- Detect NULL traps, ties, duplicate keys, join fanout, and missing dates.
- Write retention, funnel, period growth, top-N, and latest-row queries.
- Validate results with counts, reconciliations, edge cases, and samples.

- Explain a query line by line and propose relevant indexes from evidence.
- Convert findings into a concise business recommendation with guardrails.

NOTE The goal is not to remember every function. The goal is to reason correctly, find syntax efficiently, validate rigorously, and communicate what the data means.